



Module M46

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

C++11 /
C++14 Features

auto & decltype
auto
decltype

Suffix Return
Type

decltype(auto):
C++14

Module Summary

Programming in Modern C++

Module M46: C++11 and beyond: General Features: Part 1

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Weekly Recap

Module M46

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

C++11 /
C++14 Features

auto & decltype
auto
decltype

Suffix Return
Type

decltype(auto):
C++14

Module Summary

- Familiarized with I/O libraries in C and C++
- Learnt Generic Programming
- Familiarized with C++ Standard Library with specific focus to STL
- Familiarized with containers, iterators and algorithms



Module Objectives

Module M46

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

C++11 /
C++14 Features

auto & decltype
auto
decltype

Suffix Return
Type
decltype(auto):
C++14

Module Summary

- Getting familiar with C++11 and beyond: C++14, C++17, C++20, ...
- Introducing following C++11 general features:
 - auto
 - decltype
 - suffix return type (+ C++14)



Module Outline

Module M46

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

C++11 /
C++14 Features

auto & decltype
auto
decltype

Suffix Return
Type

decltype(auto):
C++14

Module Summary

- 1 Weekly Recap
- 2 Major C++11 / C++14 Features
- 3 auto & decltype
 - auto
 - decltype
- 4 Suffix Return Type
 - decltype(auto): C++14
- 5 Module Summary



Major C++11 / C++14 Features

Module M46

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

C++11 /
C++14 Features

auto & decltype
auto
decltype

Suffix Return
Type

decltype(auto):
C++14

Module Summary

Major C++11 / C++14 Features

Sources:

- [C++11 Language Extensions — General Features](#), [isocpp.org](#)
- [C++11](#), [cppreference.com](#)
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses
- [How is C++ in 2019?](#), Quora, 2019
- [C++20/17/14/11](#), [github](#)



C++ Standards

Module M46

Partha Pratim Das

Weekly Recap

Objectives & Outlines

C++11 / C++14 Features

auto & decltype
auto
decltype

Suffix Return Type

decltype(auto):
C++14

Module Summary

C++98	C++11	C++14	C++17	C++20
1998	2011	2014	2017	2020
Templates	Move Semantics	Reader-Writer Locks	Fold Expressions	Coroutines
STL with Containers and Algorithms	Unified Initialization	Generic Lambda Functions	constexpr if	Modules
Strings	auto and decltype		Structured Binding	Concepts
I/O Streams	Lambda Functions		std::string_view	Ranges Library
	constexpr		Parallel Algorithms of the STL	
	Multi-threading and Memory Model		File System Library	
	Regular Expressions		std::any, std::optional, and std::variant	
	Smart Pointers			
	Hash Tables			
	std::array			
ISO/IEC 14882:1998	ISO/IEC 14882:2011	ISO/IEC 14882:2014	ISO/IEC 14882:2017	ISO/IEC 14882:2020

Fixes on C++98: C++03: ISO/IEC 14882:2003, 2003
Latest Version as of Sep-21: C++20: ISO/IEC 14882:2020, 2020

Partha Pratim Das



Major C++11 Features: Core Language Features

Module M46

Partha Pratim Das

Weekly Recap

Objectives & Outlines

C++11 / C++14 Features

auto & decltype

auto
decltype

Suffix Return Type

decltype(auto);
C++14

Module Summary

- **auto** and **decltype**
- trailing (suffix) return type
- list initialization (**initializer_list**)
- uniform initialization: brace-or-equal initializers
- **enum class**: scoped enums
- **constexpr** and literal types
- **noexcept** specifier and operator
- **nullptr**
- **defaulted** and **deleted** functions
- delegating and inherited constructors
- **range-for** (based on Boost)
- **static_assert** (based on Boost)
- Unicode string literals
- user-defined literals
- **rvalue references** and **move semantics**
- **lambda expressions**
- **concurrency support**
- GC interface (removed in **C++23**)
- **long long**, **char16_t** and **char32_t**
- **final** and **override**
- type aliases
- variadic templates
- generalized (non-trivial) unions
- generalized PODs (trivial types and standard-layout types)
- attributes
- **alignof** and **alignas**



Major C++11 Features: Library features

Module M46

Partha Pratim Das

Weekly Recap

Objectives & Outlines

C++11 / C++14 Features

auto & decltype

auto

decltype

Suffix Return Type

decltype(auto): C++14

Module Summary

Header

- `<array>`
- `<atomic>`
- `<cfenv>`
- `<chrono>`
- `<cinttypes>`
- `<condition_variable>`
- `<cstdint>`
- `<cuchar>`
- `<forward_list>`
- `<future>`
- `<initializer_list>`
- `<mutex>`
- `<random>`
- `<ratio>`
- `<regex>`
- `<scoped_allocator>`
- `<system_error>`
- `<thread>`
- `<tuple>`
- `<typeindex>`
- `<type_traits>`
- `<unordered_map>`
- `<unordered_set>`

Library features

- atomic operations library
- `emplace()` and other use of rvalue references throughout all parts of the existing library
- `std::unique_ptr`
- `std::move_iterator`
- `std::initializer_list`
- stateful and scoped allocators
- `std::forward_list`
- chrono & ratio library
- new algorithms:
 - `std::all_of`, `std::any_of`, `std::none_of`,
 - `std::find_if_not`, `std::copy_if`, `std::copy_n`,
 - `std::move`, `std::move_backward`,
 - `std::random_shuffle`, `std::shuffle`,
 - `std::is_partitioned`, `std::partition_copy`, `std::partition_point`,
 - `std::is_sorted`, `std::is_sorted_until`, `std::is_heap`, `std::is_heap_until`,
 - `std::minmax`, `std::minmax_element`,
 - `std::is_permutation`,
 - `std::iota`,
 - `std::uninitialized_copy_n`
- Unicode conversion facets
- thread library
- `std::function`
- `std::exception_ptr`
- `std::error_code` and `std::error_condition`
- iterator improvements: `std::begin`, `std::end`, `std::next`, `std::prev`
- Unicode conversion functions



Module M46

Partha Pratim Das

Weekly Recap

Objectives & Outlines

C++11 / C++14 Features

auto & decltype

auto
decltype

Suffix Return Type

decltype(auto):
C++14

Module Summary

Core Language

- Binary literals
- Generalized return type deduction
- `decltype(auto)`
- Generalized lambda captures
- Generic lambdas
- Variable templates
- Extended `constexpr`
- The `[ [deprecated] ]` attribute
- Digit separators

Library

- Shared locking
- User-defined literals for `std::` types
- `make_unique`
- Type transformation `_t` aliases



auto & decltype

Module M46

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

C++11 /
C++14 Features

auto & decltype

auto
decltype

Suffix Return
Type

decltype(auto):
C++14

Module Summary

auto & decltype

Sources:

- [auto and decltype](http://autoanddecltype.org) isocpp.org
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses
- [Placeholder type specifiers \(since C++11\)](#) and [decltype specifier](#), cppreference.com
- [C++ auto and decltype Explained](#)
- GETTING TYPE NAME AT COMPILE TIME, Adam Badura, code::dive conference 2018, [Video](#), [Presentation](#), [Library](#)



auto

Module M46

Partha Pratim Das

Weekly Recap

Objectives & Outlines

C++11 / C++14 Features

auto & decltype
auto
decltype

Suffix Return Type

decltype(auto): C++14

Module Summary

- In C++03, `auto` designated an object with *automatic storage type*. That is now *deprecated*. We must specify the type of an object at declaration though the declaration may include an initializer with type

- In C++11 `auto` variables get the type from their *initializing expression*:

```
auto x1 = 10;           // x1: int
```

```
std::map<int, std::string> m;
```

```
auto i1 = m.begin();    // i1: std::map<int, std::string>::iterator
```

- `const/volatile` and `reference/pointer` adornments may be added:

```
const auto *x2 = &x1;    // x2: const int*
```

```
const auto& i2 = m;      // i2: const std::map<int, std::string>&
```

- To get a `const_iterator`, use `cbegin` (or `cend`, `crbegin`, and `crend`) container function:

```
auto ci = m.cbegin();    // ci: std::map<int, std::string>::const_iterator
```

- Type deduction for `auto` is akin to that for template parameters:

```
template<typename T> void f(T t);
```

```
f(expr);           // deduce T's type from expr
```

```
auto v = expr;     // do essentially the same thing for v's type
```



auto

Module M46

Partha Pratim Das

Weekly Recap

Objectives & Outlines

C++11 / C++14 Features

auto & decltype
auto
decltype

Suffix Return Type
decltype(auto);
C++14

Module Summary

- For variables *not explicitly* declared to be a *reference*:
 - Top-level **consts** / **volatiles** in the initializing type are *ignored*
 - Array and function names* in initializing types decay to *pointers*

```
const std::list<int> li;  
auto v1 = li;           // v1: std::list<int>  
auto& v2 = li;          // v2: const std::list<int>&
```

```
float data[BufSize];  
auto v3 = data;         // v3: float*  
auto& v4 = data;        // v4: float (&)[BufSize]
```

- Both *direct and copy initialization syntax* are permitted
- ```
auto v1(expr); // direct initialization syntax
auto v2 = expr; // copy initialization syntax
```

For **auto**, both syntaxes have the same meaning

- auto** is closely related to **decltype** and has extensive use in templates and generic lambdas
- ```
template<class T, class U> void multiply(const vector<T>& vt, const vector<U>& vu) {  
    // ...  
    auto tmp = vt[i]*vu[i]; // Compiler knows the type of tmp: product of T by a U  
    // ...  
}
```



decltype

Module M46

Partha Pratim Das

Weekly Recap

Objectives & Outlines

C++11 / C++14 Features

auto & decltype
auto
decltype

Suffix Return Type

decltype(auto): C++14

Module Summary

- `decltype` yields the *type of an expression without evaluating it*

```
int x, *ptr;
decltype(x) i1;           // i1's type is int
decltype(ptr) p1;         // p1's type is int*
std::size_t sz = sizeof(decltype(ptr[44])); // sz = sizeof(int); ptr[44] not evaluated
```

- Fairly intuitive, but some quirks, for example, parentheses can matter:

```
struct S { double d; };
const S* p;
decltype(p->d) x1;         // double
decltype((p->d)) x2;       // const double&
```

- Quirks rarely relevant (and can be looked up when necessary)

- Can simplify complex type expressions

```
void f(const vector<int>& a, vector<float>& b) {
    typedef decltype(a[0]*b[0]) Tmp; // Type deonted by int * float
    for (int i=0; i<b.size(); ++i) {
        Tmp* p = new Tmp(a[i]*b[i]);
        // ...
    }
    // ...
}
```



auto / decltype: Semantic Differences

- **auto** and **decltype** both infer types from expressions; but they semantically differ:

```
#include <iostream>
```

```
int main() {  
    int a = 5;           // int  
    int& b = a;          // int&  
    const int c = 7;     // const int  
    const int& d = c;    // const int&  
  
    // auto never deduces adornments like cv-qualifier or reference  
    auto a_auto = a;     // int  
    auto b_auto = b;     // int  
    auto c_auto = c;     // int  
    auto d_auto = d;     // int  
  
    // cv-qualifier or reference needs to be explicitly added  
    auto& b_auto_ref = a; // int&  
    const auto c_auto_const = a; // const int  
  
    // decltype deduces the complete type of the expression  
    decltype(a) a_dt;      // int           // [C++14] decltype(auto) a_dt_auto = a; // int  
    decltype(b) b_dt = b;  // int&          // [C++14] decltype(auto) b_dt_auto = b; // int&  
    decltype(c) c_dt = c;  // const int     // [C++14] decltype(auto) c_dt_auto = c; // const int  
    decltype(d) d_dt = d;  // const int&    // [C++14] decltype(auto) d_dt_auto = d; // const int&  
}
```



auto / decltype: Determining compiler-deduced types in C++

Module M46

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

C++11 /
C++14 Features

auto & decltype
auto
decltype

Suffix Return
Type

decltype(auto):
C++14

Module Summary

- Compiler deduces types of expressions in various contexts:
 - In C++03, types are inferred for **implicit conversions**, **templates**, etc.
 - In C++11, in addition, types are inferred for **auto** and **decltype**
- **How can we know the type deduced by the compiler?**
 - In C++ type is inferred at compiler time¹ - no support to know the inferred type
 - Debug in an IDE and check the type. This is possible only if the program compiles
 - Use compiler errors: Errors shown for: [Programiz - C++ Online Compiler]
 - ▷ Incomplete template: [Determining types deduced by the compiler in C++]

```
template<typename T> class KnowType;  
  
int arr[] = { 1, 2, 3 }; // int [3]  
KnowType<decltype(arr)> arr_type; // error: aggregate 'KnowType<int [3]> arr_type'  
// has incomplete type and cannot be defined
```
 - ▷ Incomplete type: [Using 'auto' type deduction - how to find out what type the compiler deduced?]

```
int arr[] = { 1, 2, 3 }; // int [3]  
decltype(arr)::_;  
// error: decltype evaluates to 'int [3]', which  
// is not a class or enumeration type
```

¹C++ is statically typed (except for dynamic polymorphism where there is **typeid** support for type)



auto / decltype: Determining compiler-deduced types in C++

Module M46

Partha Pratim Das

Weekly Recap

Objectives & Outlines

C++11 / C++14 Features

auto & decltype

auto

decltype

Suffix Return Type

decltype(auto): C++14

Module Summary

- We may also use `typeid` operator to know the type
 - Not a good idea as `typeid` is meant for dynamic type
 - The name of type returned by `typeid` is encoded

```
#include <bits/stdc++.h> // includes all standard library, but is not a standard header file of GNU C++
                        // DO NOT USE

using namespace std;

int main() {
    int x;                // int
    char y;               // char
    cout << typeid(x).name() << endl;    // i
    cout << typeid(y).name() << endl;    // c

    vector<int> vi;        // std::vector<int>
    vector<double> vd;     // std::vector<double>
    cout << typeid(vi).name() << endl;    // St6vectorIiSaIiEE
    cout << typeid(vd).name() << endl;    // St6vectorIdSaIdEE

    auto it = vi.begin();  // std::vector<int>::iterator
    decltype(vi.cbegin()) cit = vi.cbegin(); // std::vector<int>::const_iterator
    cout << typeid(it).name() << endl;    // N9__gnu_cxx17__normal_iteratorIPiSt6vectorIiSaIiEEEE
    cout << typeid(cit).name() << endl;    // N9__gnu_cxx17__normal_iteratorIPKiSt6vectorIiSaIiEEEE
}
```




Suffix / Trailing Return Type

Module M46

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

C++11 /
C++14 Features

auto & decltype
auto
decltype

Suffix Return
Type

decltype(auto):
C++14

Module Summary

Suffix / Trailing Return Type

Sources:

- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses
- [Function declaration](#), cppreference.com
- [When to use decltype\(auto\) versus auto?](#), cplusplus.com



Suffix / Trailing Return Type

Module M46

Partha Pratim Das

Weekly Recap

Objectives & Outlines

C++11 / C++14 Features

auto & decltype
auto
decltype

Suffix Return Type

decltype(auto): C++14

Module Summary

- We really need `decltype` if we need a type for something *that is not a variable*, such as a *return type*. Consider:

```
template<class T, class U>  
??? mul(T x, U y) { return x*y; }
```

- How to write the *return type*? It is the *type of x*y* – but how can we say that? Use `decltype`?

```
template<class T, class U>  
decltype(x*y) mul(T x, U y) { return x*y; } // scope problem! types of x and y not known
```

- That won't work because `x` and `y` are not in scope. So:

```
template<class T, class U>  
(decltype*(T*)(0)**(U*)(0)) mul(T x, U y) { return x*y; } // ugly! and error prone
```

- Put the return type where it belongs, after the arguments:

```
template<class T, class U>  
auto mul(T x, U y) -> decltype(x*y) { return x*y; }
```

We use the notation `auto` to mean *return type to be deduced or specified later*



Suffix / Trailing Return Type

Module M46

Partha Pratim Das

Weekly Recap

Objectives & Outlines

C++11 / C++14 Features

auto & decltype
auto
decltype

Suffix Return Type

decltype(auto): C++14

Module Summary

- The *suffix syntax* is not primarily about *templates* and *type deduction*, it is really about *scope*

```
struct List {  
    struct Link { /* ... */ };  
    Link* erase(Link* p); // remove p and return the link before p  
    // ...  
};  
List::Link* List::erase(Link* p) { /* ... */ }
```

- The first `List::` is necessary only because the scope of `List` is not entered until the second `List::`. Better:
`auto List::erase(Link* p) -> Link* { /* ... */ } // No explicit qualification for Links`
- To declare objects, `decltype` can replace `auto`, but more verbosely:
`std::vector<std::string> vs;
auto i = vs.begin();
decltype(vs.begin()) i = vs.begin();`
- Only `decltype` solves the template-return-type problem in C++11 (by Perfect Forwarding)
- `auto` is for everybody. `decltype` is primarily for template authors



Suffix / Trailing Return Type: C++14

Module M46

Partha Pratim Das

Weekly Recap

Objectives & Outlines

C++11 / C++14 Features

auto & decltype
auto
decltype

Suffix Return Type

decltype(auto): C++14

Module Summary

- In C++11, we use suffix return type to specify return type of templates to be inferred:

```
template<class T, class U>  
auto mul(T x, U y) -> decltype(x*y) { return x*y; }
```

This is unclean because the return expression has to be *repeated* within *decltype*

- In C++14, suffix type can be skipped and the return type is deduced directly:

```
template<class T, class U>  
auto mul(T x, U y) { return x*y; }
```

For compatibility, it still supports the suffix return type. Hence, the following is still valid:

```
template<class T, class U>  
auto mul(T x, U y) -> decltype(x*y) { return x*y; }
```

- C++14, further introduces *decltype(auto)* for deducing the return type by the semantics of *decltype* and not the semantics of *auto*. *No suffix return type is allowed here*

```
template<class T, class U>  
decltype(auto) mul(T x, U y) { return x*y; }
```

- We present an example to highlight the differences between *auto* and *decltype(auto)*



Suffix / Trailing Return Type: C++14: auto / decltype(auto)

Module M46

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

C++11 /
C++14 Features

auto & decltype
auto
decltype

Suffix Return
Type

decltype(auto):
C++14

Module Summary

```
#include <iostream>
```

```
// returns prvalue: plain auto never deduces to a reference. prvalue is a pure rvalue - TBD later  
template<typename T> auto foo(T& t) { return t.value(); }
```

```
// return lvalue: auto& always deduces to a reference  
template<typename T> auto& bar(T& t) { return t.value(); }
```

```
// return prvalue if t.value() is an rvalue  
// return lvalue if t.value() is an lvalue  
// decltype(auto) has decltype semantics (without having to repeat the expression)  
template<typename T> decltype(auto) foobar(T& t) { return t.value(); }
```

```
int main() {  
    struct A { int i = 0 ; int& value() { return i ; } } a;  
    struct B { int i = 0 ; int value() { return i ; } } b;  
  
    foo(a) = 20; // *** error: expression evaluates to prvalue of type int  
    foo(b);      // fine: expression evaluates to prvalue of type int  
  
    bar(a) = 20; // fine: expression evaluates to lvalue of type int&  
    bar(b);      // *** error: auto& always deduces to a reference (int&) - bar(b) needs an initializer  
  
    foobar(a) = 20; // fine: expression evaluates to lvalue of type int&  
    foobar(b);      // fine: expression evaluates to prvalue of type int  
}
```



Suffix / Trailing Return Type: `auto` / `decltype(auto)`

Module M46

Partha Pratim Das

Weekly Recap

Objectives & Outlines

C++11 / C++14 Features

`auto` & `decltype`
`auto`
`decltype`

Suffix Return Type

`decltype(auto)`: C++14

Module Summary

• Recommendations

- `auto`
 - ▷ Use `auto` to return a *prvalue with type deduction*
 - ▷ Use `auto&` or `const auto&` to return an *lvalue with type deduction*
- `decltype(auto)`
 - ▷ Use `decltype(auto)` to write *forwarding templates*
 - ▷ Using `decltype(auto)`, the return type is as what would be obtained if the expression used in the *return statement were wrapped in decltype*
 - ▷ Without `decltype(auto)`, the deduction follows rules of *template argument deduction*

• Summary

```
auto foo() { // rt = int
    int x = 0; return (x); // returning local variable by value. Fine
}

decltype(auto) bar() { // rt = int&
    int x = 0; return (x); // returning local variable by reference. Trouble
}
```

Source: [When to use `decltype\(auto\)` versus `auto`](#)

Programming in Modern C++

Partha Pratim Das

M46.22



Module Summary

Module M46

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

C++11 /
C++14 Features

auto & decltype
auto
decltype

Suffix Return
Type
decltype(auto):
C++14

Module Summary

- Introduced following **C++11** general features:
 - auto
 - decltype
 - suffix return type (+ **C++14**)



Module M47

Partha Pratim
Das

Objectives &
Outlines

Initializer Lists

`initializer_list`

Overload Resolution

`{ }`-Initializers and
`auto`

Uniform
Initialization

Syntax
Semantics

Range-for

Module Summary

Programming in Modern C++

Module M47: C++11 and beyond: General Features: Part 2

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M47

Partha Pratim
Das

Objectives &
Outlines

Initializer Lists

`initializer_list`

Overload Resolution

`{ }`-Initializers and
`auto`

Uniform
Initialization

Syntax

Semantics

Range-for

Module Summary

- Introduced following **C++11** general features:
 - `auto`
 - `decltype`
 - suffix return type (+ **C++14**)



Module Objectives

Module M47

Partha Pratim
Das

Objectives &
Outlines

Initializer Lists

initializer_list

Overload Resolution

{ }-Initializers and
auto

Uniform
Initialization

Syntax

Semantics

Range-for

Module Summary

- Introducing following **C++11** general features:
 - Initializer List
 - Uniform Initialization
 - Range for Statement



Module Outline

Module M47

Partha Pratim Das

Objectives & Outlines

Initializer Lists

`initializer_list`

Overload Resolution

`{}`-Initializers and `auto`

Uniform Initialization

Syntax

Semantics

Range-for

Module Summary

- 1 Initializer Lists
 - `initializer_list`
 - Overload Resolution
 - `{}`-Initializers and `auto`
- 2 Uniform Initialization: Syntax and Semantics
 - Syntax
 - Semantics
- 3 Range-for Statement
- 4 Module Summary



Initializer Lists

Module M47

Partha Pratim
Das

Objectives &
Outlines

Initializer Lists

`initializer_list`

Overload Resolution

`{ }`-Initializers and
`auto`

Uniform
Initialization

Syntax

Semantics

Range-for

Module Summary

Initializer Lists

Sources:

- [Initializer lists](http://isocpp.org), isocpp.org
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses
- [List-initialization \(since C++11\)](#) and [Constructors and member initializer lists](#), cppreference.com



Initializer Lists

Module M47

Partha Pratim
Das

Objectives &
Outlines

Initializer Lists

initializer_list

Overload Resolution

{ }-Initializers and
auto

Uniform
Initialization

Syntax
Semantics

Range-for

Module Summary

- Consider:

```
vector<double> v = { 1, 2, 3.456, 99.99 }; // list of doubles
list<pair<string,string>> languages = { // list of pairs of strings
    {"Nygaard", "Simula"}, {"Richards", "BCPL"}, {"Ritchie", "C"}
};
map<vector<string>,vector<int>> years = { // list of vector<string>s and vector<int>s
    { {"Maurice", "Vincent", "Wilkes"}, {1913, 1945, 1951, 1967, 2000} },
    { {"Martin", "Ritchards"}, {1982, 2003, 2007} },
    { {"David", "John", "Wheeler"}, {1927, 1947, 1951, 2004} }
};
```

- Initializer lists are not just for arrays. The mechanism for accepting a `{}`-list (*braced list*) is a *function* (often a *constructor*) accepting an argument of type `std::initializer_list<T>`:

```
void f(initializer_list<int>);
f({1,2});
f({23, 345, 4567, 56789});
f({}); // the empty list
f{1,2}; // error: function call () missing
years.insert({{"Bjarne", "Stroustrup"}, {1950, 1975, 1985}});
```



Initializer Lists

Module M47

Partha Pratim Das

Objectives & Outlines

Initializer Lists

initializer_list

Overload Resolution

{ }-Initializers and auto

Uniform Initialization

Syntax Semantics

Range-for

Module Summary

- The initializer list can be of *arbitrary length*, but must be *homogeneous* (*all elements must be of the template argument type, T, or convertible to T*). An *initializer-list constructor* may be implemented as:

```
template<class E> class vector { public:
    vector(std::initializer_list<E> s) {           // initializer-list constructor
        reserve(s.size());                       // get the right amount of space
        uninitialized_copy(s.begin(), s.end(), elem); // init. elements (in elem[0:s.size()))
        sz = s.size();                           // set vector size
    }
};
```

- The distinction between *direct* and *copy initialization* is maintained for {}-initialization, but is *less relevant*. For `std::vector` with an *explicit* ctor from `int` and an *initializer_list* ctor:

```
vector<double> v1(7);           // okay: v1 has 7 elements
v1 = 9;                         // error: no conversion from int to vector
vector<double> v2 = 9;          // error: no conversion from int to vector
void f(const vector<double>&);
f(9);                           // error: no conversion from int to vector
vector<double> v1{7};            // okay: v1 has 1 element (with its value 7.0)
v1 = {9};                       // okay v1 now has 1 element (with its value 9.0)
vector<double> v2 = {9};         // okay: v2 has 1 element (with its value 9.0)
f({9});                         // okay: f is called with the list { 9 }
vector<vector<double>> vs = {
    vector<double>(10),          // okay: explicit construction (10 elements)
    vector<double>{10},         // okay explicit construction (1 element with the value 10.0)
    10                          // error: vector's constructor is explicit
};
```



Initializer Lists

Module M47

Partha Pratim Das

Objectives & Outlines

Initializer Lists

initializer_list

Overload Resolution

{ }-Initializers and auto

Uniform Initialization

Syntax Semantics

Range-for

Module Summary

- The function can access the `initializer_list` as an *immutable sequence*. For example:

```
void f(initializer_list<int> args) {  
    for (auto p = args.begin(); p != args.end(); ++p) cout << *p << "\n";  
}
```
- `std::initializer_list` stores initializer values in an array and offers these member functions:
 - `size` // # of elements in the array
 - `begin` // ptr to first array element
 - `end` // ptr to one-beyond-last array element
- A constructor that takes a single argument of type `std::initializer_list` is called an **initializer-list constructor**
- The STL containers, `string`, and `regex` have initializer-list constructors, assignment, etc. An initializer-list can be used as a range, for example, in a *range for* statement (TBD later).
- The initializer lists are part of the scheme for *uniform and general initialization*. They also prevent *narrowing*
- Usually initializing using `{ }` is preferred instead of `( )` unless:
 - The code is shared with a C++03 compiler or
 - There is a need to use `( )` to call a non-`initializer_list` overloaded constructor (rare)



Initializer Lists: `std::initializer_list`

- `std::initializer_list` looks something like: [Initializer Lists in C++ – `std::initializer_list`]

```
typedef unsigned int size_t; // #include <bits/C++config.h>
namespace std {
    template<class _E> class initializer_list { // initializer list
    public:
        typedef _E value_type;
        typedef const _E& reference;
        typedef const _E& const_reference;
        typedef size_t size_type;
        typedef const _E* iterator;
        typedef const _E* const_iterator;
    private:
        iterator _M_array;
        size_type _M_len;
        // The compiler can call a private constructor
        // constexpr defines compile-time constant expressions - TBD later
        constexpr initializer_list(const_iterator __a, size_type __l): _M_array(__a), _M_len(__l) { }
    public:
        constexpr initializer_list() noexcept: _M_array(0), _M_len(0) { }
        constexpr size_type size() const noexcept { return _M_len; } // Number of elements
        constexpr const_iterator begin() const noexcept { return _M_array; } // First element
        constexpr const_iterator end() const noexcept { return begin()+size(); } // One past last element
    };
};
```




Initializer Lists: initializer-list constructor

Module M47

Partha Pratim
Das

Objectives &
Outlines

Initializer Lists

initializer_list

Overload Resolution

{ }-Initializers and
auto

Uniform
Initialization

Syntax
Semantics

Range-for

Module Summary

```
#include <iostream>
#include <string>
#include <vector>
#include <initializer_list>

template <typename T> // T is the type of initializer_list elements
class MyClass { std::vector<T> elems; /* vector to keep initialized values*/ public:
    // Default constructor
    MyClass(): elems({-1}) { std::cout << "Default Ctor: "; ShowElements(); }
    // Parametrized constructor
    MyClass(int b): elems({b}) { std::cout << "Parametrized Ctor: "; ShowElements(); }
    // Constructor using std::initializer_list
    MyClass(std::initializer_list<T> init_list): elems({init_list}) { // Using parenthesis
        // We can directly iterate on init_list as we do over elems
        std::cout << "Initializer List Ctor: "; ShowElements();
    }
    // Mixed Constructor
    MyClass(int i, std::initializer_list<T> init_list): elems{init_list} { // Without using parenthesis
        std::cout << "Mixed Ctor: " << i << ", "; ShowElements();
    }
    void ShowElements() /* Display the elements of elems */ { std::cout << "{ ";
        for (auto it = elems.begin(); it != elems.end(); ++it) std::cout << *it << ' ';
        std::cout << "}\n";
    }
};
```

Programming in Modern C++



Initializer Lists: initializer-list constructor

Module M47

Partha Pratim
Das

Objectives &
Outlines

Initializer Lists

initializer_list

Overload Resolution

{ }-Initializers and
auto

Uniform
Initialization

Syntax
Semantics

Range-for

Module Summary

```
// template <typename T> class MyClass { std::vector<T> elems; public:
//     MyClass(); // Default constructor
//     MyClass(int b); // Parametrized constructor
//     MyClass(std::initializer_list<T> init_list); // Constructor using std::initializer_list
//     MyClass(int i, std::initializer_list<T> init_list); // Mixed Constructor
//     void ShowElements();
// };

int main() {
    MyClass<int> my_obj; // my_obj{} */ // Default Ctor: { -1 }
    MyClass<int> my_obj_i = MyClass<int>(500); // my_obj_i(500) */ // Parametrized Ctor: { 500 }
    MyClass<int> my_obj_il = MyClass<int>{500}; // my_obj_il{500} */ // Initializer List Ctor: { 500 }

    // initializer_list objects: std::initializer_list<int>
    auto init_list = { 1, 2, 3, 4, 5 };
    // May use init_list for { init_list }
    MyClass<int> my_obj_il_int = { init_list }; // Initializer List Ctor: { 1 2 3 4 5 }

    // initializer_list object
    std::initializer_list<std::string> il = { "Hello", "from", "PPD" };
    // May use il for { il }
    MyClass<std::string> my_obj_il_string = { il }; // Initializer List Ctor: { Hello from PPD }

    MyClass<std::string> my_obj_m = { 5, { "Thank", "You" } }; // Mixed Ctor: 5, { Thank You }
}
```



Initializer Lists: Overload Resolution

Module M47

Partha Pratim Das

Objectives & Outlines

Initializer Lists

initializer_list

Overload Resolution

{ }-Initializers and auto

Uniform Initialization

Syntax

Semantics

Range-for

Module Summary

- Constructor with `std::initializer_list` parameter prefers `{}`-delimited arguments

```
class Widget { public:
    Widget(std::initializer_list<double> values); // #1
    Widget(double value, double uncertainty);    // #2
    ...
};
double d1, d2;
...
Widget w1 { d1, d2 }; // calls #1
Widget w2(d1, d2);   // calls #2
```

- Choose carefully between `{}` and `()` when initializing objects!

```
template <class T, class Allocator = allocator<T> > // from the C++11 standard
class vector { public: ...
    vector(size_type n, const T& value, const Allocator& = Allocator());
    vector(initializer_list<T>, const Allocator& = Allocator());
    ...
};
std::vector<int> v1(10, 5); // v1.size() == 10, all values == 5
std::vector<int> v2{10, 5}; // v2.size() == 2, values == {10, 5}
```



Initializer Lists: Overload Resolution

Module M47

Partha Pratim
Das

Objectives &
Outlines

Initializer Lists
initializer_list

Overload Resolution
{ }-Initializers and
auto

Uniform
Initialization

Syntax
Semantics

Range-for

Module Summary

- Given multiple `std::initializer_list` candidates, best match is determined by worst element conversion:

```
class Widget { public:  
    Widget(std::initializer_list<int>);           // #1  
    Widget(std::initializer_list<double>);        // #2  
    Widget(std::initializer_list<std::string>);   // #3  
    ...  
};  
Widget w1 { 1, 2.0, 3 };           // int => double same rank as double => int, so ambiguous  
Widget w2 { 1.0f, 2.0, 3.0 };      // float => double better than float => int, so calls #2  
std::string s  
Widget w3 { s, "Init", "Lists" };  // calls #3
```

- If best match involves a narrowing conversion, call is invalid:

```
class Widget { public:  
    Widget(std::initializer_list<int>);           // #1  
    Widget(int, int, int);                       // #2  
};  
Widget w1 { 1, 2.0, 3 };           // Matches #1: error! double => int narrows  
Widget w2 ( 1, 2.0, 3 );          // Matches #2: okay. double => int narrowing allowed
```



Initializer Lists: Braced Initializers and auto

Module M47

Partha Pratim Das

Objectives & Outlines

Initializer Lists

initializer_list

Overload Resolution

{ }-Initializers and auto

Uniform Initialization

Syntax

Semantics

Range-for

Module Summary

- `auto` deduces `std::initializer_list` for braced initializers:
`auto i = { 2, 4, 6, 8 }; // i is std::initializer_list<int>`
- In general, templates deduce no type for braced initializers:
`template<typename T> void f(T param) { ... }
f({ 2, 4, 6, 8 }); // error! no type deduced for { 2, 4, 6, 8 }`
 - Only way that `auto` type deduction $\neq$ template type deduction
- Especially for single-element braced initializers, this can confuse:
`auto i1 = 10; // i1 is int
auto i2(10); // i2 is int
auto i3 {10}; // i3 is std::initializer_list<int>`
- Particularly when such variables interact with overload resolution:
`std::vector<int> v1(i1); // v1.size() == 10, values == 0
std::vector<int> v2(i2); // v1.size() == 10, values == 0
std::vector<int> v3(i3); // v1.size() == 1, value == 10`
- Use care when initializing `auto` variables with braced initializers!



Uniform Initialization: Syntax and Semantics

Module M47

Partha Pratim
Das

Objectives &
Outlines

Initializer Lists

`initializer_list`

Overload Resolution

`{ }`-Initializers and
`auto`

Uniform
Initialization

Syntax

Semantics

Range-for

Module Summary

Uniform Initialization: Syntax and Semantics

Sources:

- [Uniform initialization syntax and semantics](http://isocpp.org), isocpp.org
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses
- [Initialization and Constructors and member initializer lists](http://cpreference.com), cpreference.com



Uniform Initialization Syntax

Module M47

Partha Pratim Das

Objectives & Outlines

Initializer Lists

initializer_list

Overload Resolution

{ }-Initializers and auto

Uniform Initialization

Syntax

Semantics

Range-for

Module Summary

- C++03 offers multiple initialization forms
 - Initialization $\neq$ assignment. For example, `const` objects can be initialized, not assigned

- Examples:

```
const int y(5);           // direct initialization syntax
const int x = 5;          // copy initialization syntax
int arr[] = { 5, 10, 15 }; // brace initialization
struct Point1 { int x, y; };
const Point1 p1 = { 10, 20 }; // brace initialization
class Point2 { public: Point2(int x, int y); };
const Point2 p2(10, 20);    // function call syntax
```

- Containers require another container:

```
int vals[] = { 10, 20, 30 };
const std::vector<int> cv(vals, vals+3); // init from another container
```

- Member and heap arrays are impossible:

```
class Widget {
public: Widget(): data(???) {}
private: const int data[5];           // not initializable
};
const float * pData = new const float[4]; // not initializable
```



Uniform Initialization Syntax

Module M47

Partha Pratim Das

Objectives & Outlines

Initializer Lists

initializer_list

Overload Resolution

{ }-Initializers and auto

Uniform Initialization

Syntax

Semantics

Range-for

Module Summary

- Brace initialization syntax now allowed *everywhere*:

```
const int val1 {5};
const int val2 {5};
int a[] { 1, 2, val1, val1+val2 };
struct Point1 { int x, y; };
const Point1 p1 {10, 20};
class Point2 { public: Point2(int x, int y); };
const Point2 p2 {10, 20}; // calls Point2 ctor
const std::vector<int> cv { a[0], 20, val2 };
class Widget {
    public: Widget(): data {1, 2, a[3], 4, 5} {}
    private: const int data[5];
};
const float * pData = new const float[4] { 1.5, val1-val2, 3.5, 4.5 };
```

- Really, *everywhere*:

```
Point2 makePoint() { return { 0, 0 }; } // return expression; calls Point2 ctor

void f(const std::vector<int>& v); // function declaration
f({ val1, val2, 10, 20, 30 }); // function argument
```




Uniform Initialization Semantics

Module M47

Partha Pratim Das

Objectives & Outlines

Initializer Lists

`initializer_list`

Overload Resolution

`{ }`-Initializers and `auto`

Uniform Initialization

Syntax

Semantics

Range-for

Module Summary

- Semantics differ for aggregates and non-aggregates:
 - **Aggregates** (for example, arrays and structs):
 - ▷ Initialize members/elements beginning-to-end
 - **Non-aggregates**:
 - ▷ Invoke a constructor



Uniform Initialization Semantics: { }-Initializing Aggregates & Non-Aggregates

Module M47

Partha Pratim Das

Objectives & Outlines

Initializer Lists

initializer_list

Overload Resolution

{ }-Initializers and auto

Uniform Initialization

Syntax

Semantics

Range-for

Module Summary

- **Aggregates:** Initialize members/elements beginning-to-end

- Too many initializers $\Rightarrow$ *error*
- Too few initializers $\Rightarrow$ remaining objects are *value-initialized*

- ▷ Built-in types initialized to 0
- ▷ UDTs with constructors are default-constructed
- ▷ UDTs without constructors: members are value-initialized

```
struct Point1 { int x, y; };  
const Point1 p1 = { 10 };           // same as { 10, 0 }  
const Point1 p2 = { 1, 2, 3 };      // error! too many initializers
```

- ▷ `std::array` is also an aggregate:

```
long f();  
std::array<long, 3> arr = { 1, 2, f(), 4, 5 }; // error! too many initializers
```

- **Non-Aggregates:** Invoke a constructor

```
class Point2 { public: Point2(int x, int y); }; short a, b;
```

```
...
```

```
const Point2 p1 {a, b};           // same as p1(a, b)  
const Point2 p2 {10};             // error! too few ctor args  
const Point2 p3 {5, 10, 20};      // error! too many ctor args
```

```
std::vector<int> v { 1, a, 2, b, 3 }; // Holds for containers - calls a vector ctor  
std::unordered_set<float> s { 0, 1.5, 3 }; // calls an unordered_set ctor
```



Uniform Initialization Semantics

Module M47

Partha Pratim Das

Objectives & Outlines

Initializer Lists

`initializer_list`

Overload Resolution

`{ }`-Initializers and `auto`

Uniform Initialization

Syntax

Semantics

Range-for

Module Summary

- Brace-initialized variables may use `=`:

```
const int val1 = {5};
const int val2 = {5};
int a[] = { 1, 2, val1, val1+val2 };
struct Point1 { ... };
const Point1 p1 = {10, 20};
class Point2 { ... };
const Point2 p2 = {10, 20};
const std::vector<int> cv = { a[0], 20, val2 };
```

- Other uses of brace initialization cannot:

```
class Widget { // Not allowed in member initialization
public: Widget(): data = {1, 2, a[3], 4, 5} {} // error!
private: const int data[5];
};
// Not allowed in dynamic allocation
const float *pData = new const float[4] = { 1.5, val1-val2, 3.5, 4.5 }; // error!

// Not allowed in return
Point2 makePoint() { return = { 0, 0 }; } // error!

// Not allowed in function parameters
void f(const std::vector<int>& v);
f( = { val1, val2, 10, 20, 30 }); // error!
```



Uniform Initialization Semantics

Module M47

Partha Pratim Das

Objectives & Outlines

Initializer Lists

initializer_list

Overload Resolution

{ }-Initializers and auto

Uniform Initialization

Syntax Semantics

Range-for

Module Summary

- And `T var = expr` syntax cannot call `explicit` constructors:

```
class Widget {  
public:  
    explicit Widget(int);  
    ...  
};  
Widget w1(10);      // okay, direct init: explicit ctor callable  
Widget w2{10};      // okay, direct init: explicit ctor callable  
Widget w3 = 10;      // error! copy init: explicit ctor not callable  
Widget w4 = {10};    // error! copy init: explicit ctor not callable
```

- Develop the habit of using brace initialization without `=`
- Uniform initialization syntax a feature *addition*, not a replacement
 - Almost all initialization code valid in C++03 remains valid
 - ▷ Rarely a need to modify existing code



Uniform Initialization Semantics: { }-Initialization and Implicit Narrowing

Module M47

Partha Pratim
Das

Objectives &
Outlines

Initializer Lists

initializer_list

Overload Resolution

{ }-Initializers and
auto

Uniform
Initialization

Syntax
Semantics

Range-for

Module Summary

- Sole exception: implicit narrowing

- C++03 *allows it via brace initialization*, C++11 *does not*

```
struct Point { int x, y; };  
Point p1 = { 1, 2.5 };           // fine in C++03  
                                   // implicit double => int conversion  
                                   // error in C++11  
  
Point p2 = { 1, static_cast<int>(2.5) }; // fine in both C++03 and C++11
```

- Direct constructor calls and brace initialization thus differ subtly:

```
class Widget {  
    public: Widget(unsigned u);  
    ...  
};  
int i;  
...  
Widget w1(i);           // okay, implicit int => unsigned  
Widget w2{i};           // error! int => unsigned narrows  
unsigned u;  
Widget w3(u);           // fine  
Widget w4{u};           // also fine, same as w3's init.
```



Uniform Initialization Summary

Module M47

Partha Pratim Das

Objectives & Outlines

Initializer Lists

`initializer_list`

Overload Resolution

`{}`-Initializers and `auto`

Uniform Initialization

Syntax

Semantics

Range-for

Module Summary

- Brace initialization syntax now available everywhere
 - *Aggregates* initialized *top-to-bottom / front-to-back*
 - *Non-aggregates* initialized via *constructor*
- *Implicit narrowing* not allowed.
- `std::initializer_list` parameters allow *initialization* lists to be passed to functions
 - Not actually limited to initialization (for example, `std::vector::insert`)
- Choose carefully between `{}` and `()` when initializing objects
 - Remember that `auto + { expr }` yields `std::initializer_list`



Range-for Statement

Module M47

Partha Pratim
Das

Objectives &
Outlines

Initializer Lists

`initializer_list`

Overload Resolution

`{ }`-Initializers and
`auto`

Uniform
Initialization

Syntax

Semantics

Range-for

Module Summary

Range-for Statement

Sources:

- [Range-for statement](http://isocpp.org), isocpp.org
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses
- [Range-based for loop in C++](#)



Ways of traversing a vector: Recap (Module 44)

Module M47

Partha Pratim
Das

Objectives &
Outlines

Initializer Lists

initializer_list

Overload Resolution

{ }-Initializers and
auto

Uniform
Initialization

Syntax
Semantics

Range-for

Module Summary

- Let us revisit ways for the traversal of a vector as a sample container:

```
vector<int> v;
```

```
// subscript style
```

```
for(int i = 0; i < v.size(); ++i) { /* use v[i] */ } // native int for size
```

```
for(vector<T>::size_type i = 0; i < v.size(); ++i) { /* use v[i] */ } // correct size_type
```

```
// iterator style
```

```
for(vector<T>::iterator p = v.begin(); p != v.end(); ++p) { /* use *p */ } // verbose
```

```
for(vector<T>::value_type x : v) { /* use x read-only */ } // range for [C++11]
```

```
for(auto& x : v) { /* use x read-write */ } // range for [C++11]
```

- Comparing *subscript* and *iterator* styles:
 - The *subscript style* is used in essentially every language
 - The *subscript style* does not work for lists and non-linear data structures (in C++ and in most languages)
 - The *iterator style* is used in C (pointers only) and C++
 - The *iterator style* is used for standard library algorithms
 - While both styles work for vectors, *iterator style* is more generic – works for all sequences



Range-for Statement

Module M47

Partha Pratim
Das

Objectives &
Outlines

Initializer Lists

initializer_list

Overload Resolution

{ }-Initializers and
auto

Uniform
Initialization

Syntax
Semantics

Range-for

Module Summary

- A **range for statement** allows us to iterate through a **range**, which is anything we can iterate through like an STL-sequence defined by a **begin()** and **end()**
- All standard containers can be used as a range, as can a **std::string**, an initializer list, an array, a valarray, and any UDT that supports **begin()** and **end()**, for example, an **istream**:

```
void f(vector<double>& v) {  
    for (auto x : v) cout << x << endl; // auto is vector<double>::value_type  
    for (auto& x : v) ++x; // using a reference to allow us to change the value  
}
```

- A range for is read as **for all x in v** going through starting with **v.begin()** and iterating to **v.end()**:

```
for (const auto x : { 1, 2, 3, 5, 8, 13, 21, 34 })  
    cout << x << endl;
```

- **volatile** may also be used:

```
for (volatile int i : v) someOtherFunc(i); // or volatile auto i
```

- The **begin()** (and **end()**) can be a member to be called as **x.begin()** or a free-standing function to be called as **begin(x)**. *The member version takes precedence*



Range-for Statement: Example

Module M47

Partha Pratim
Das

Objectives &
Outlines

Initializer Lists
initializer_list

Overload Resolution
{ }-Initializers and
auto

Uniform
Initialization

Syntax
Semantics

Range-for

Module Summary

```
// Illustration of range-for loop using CPP code
#include <iostream>
#include <vector>
#include <map>
int main() {
    // Driver
    std::vector<int> v = { 0, 1, 2, 3, 4, 5 }; // Iterating over whole array
    for (auto i : v) std::cout << i << ' '; std::cout << std::endl;

    // the initializer may be a braced-init-list
    for (int n : { 0, 1, 2, 3, 4, 5 }) std::cout << n << ' '; std::cout << std::endl;

    int a[] = { 0, 1, 2, 3, 4, 5 }; // Iterating over array
    for (int n : a) std::cout << n << ' '; std::cout << std::endl;

    // Just running a loop for every array element
    for (int n : a) std::cout << "In loop" << ' '; std::cout << std::endl;

    std::string str = "PPD"; // Printing string characters
    for (char c : str) std::cout << c << ' '; std::cout << std::endl;

    std::map <int, int> MAP({{1, 1}, {2, 4}, {3, 9}}); // Printing keys and values of a map
    for (auto i : MAP) std::cout << ' ' << i.first << ", " << i.second << std::endl;
}
```



Module Summary

Module M47

Partha Pratim
Das

Objectives &
Outlines

Initializer Lists

`initializer_list`

Overload Resolution

`{ }`-Initializers and
`auto`

Uniform
Initialization

Syntax

Semantics

Range-for

Module Summary

- Introduced following **C++11** general features:
 - Initializer List
 - Uniform Initialization
 - Range for Statement



Module M48

Partha Pratim
Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

Programming in Modern C++

Module M48: C++11 and beyond: General Features: Part 3

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M48

Partha Pratim
Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

- Introduced following **C++11** general features:
 - Initializer List
 - Uniform Initialization
 - Range for Statement



Module Objectives

Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

- Introducing following **C++11** general features:
 - constexpr (+ **C++14**)
 - noexcept
 - nullptr
 - Inline namespace
 - static_assert
 - User-defined Literals (+ **C++14**)
 - Digit Separators and Binary Literals (+ **C++14**)
 - Raw String Literals
 - Unicode Support
 - Memory Alignment
 - Attributes (+ **C++14**)



Module Outline

Module M48

Partha Pratim
Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

- 1 **constexpr**: Evaluate constant expressions at compile-time
- 2 **noexcept**: To prevent Exception Propagation
- 3 **nullptr**: null Pointer Literal
- 4 **Inline namespaces**: Efficient Version Management
- 5 **static_assert**: Compile-time Assertions
- 6 **User-defined Literals**: UDTs closer to Built-in Types
- 7 **Digit Separators and Binary Literals**
- 8 **Raw String Literals**
- 9 **Unicode Support**
- 10 **Memory Alignment**
- 11 **Attributes**
- 12 **Module Summary**



constexpr: Evaluate constant expressions at compile-time

Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

Sources:

- [constexpr](#), [isocpp.org](#)
- [Demystifying constexpr](#), 2016
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses
- [Understanding constexpr specifier in C++](#), [geeksforgeeks.org](#)
- [Difference between 'constexpr' and 'const'](#), [stackoverflow.com](#), 2013
- [C++20 consteval specifier](#)

constexpr: Evaluate constant expressions at compile-time



constexpr

Module M48

Partha Pratim
Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

- The `constexpr` mechanism
 - provides more *general constant expressions*
 - allows constant expressions involving *user-defined types*
 - provides a way to guarantee that an *initialization is done at compile time*

```
enum Flags { good = 0, fail = 1, bad = 2, eof = 4 };  
constexpr int operator|(Flags f1, Flags f2)  
{ return Flags(int(f1) | int(f2)); }
```

```
void f(Flags x) {  
    switch (x) {  
        case bad:      /* ... */ break;  
        case eof:      /* ... */ break;  
        case bad|eof:  /* ... */ break;  
        default:       /* ... */ break;  
    }  
}
```



constexpr

Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

- Here `constexpr` says that the function must be of a simple form so that it *can be evaluated at compile time if given constant expressions arguments*
- In addition to be able to evaluate expressions at compile time, we want to be able to require expressions to be evaluated at compile time
- `constexpr` in front of a variable definition does that (and implies `const`):

```
constexpr int x1 = bad|eof;    // okay

void f(Flags f3) {
    constexpr int x2 = bad|f3; // error: cannot evaluate at compile time
    int x3 = bad|f3;           // okay
}
```

- Recall the use of `constexpr` in `std::initializer_list` in **Module 47**



constexpr

Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

- Typically we want the compile-time evaluation guarantee for *global or namespace objects*, often for objects we want to place in *read-only storage*
- This also works for objects for which the constructors are simple enough to be **constexpr** and expressions involving such objects:

```
struct Point {  
    int x, y;  
    constexpr Point(int xx, int yy) : x(xx), y(yy) { }  
};
```

```
constexpr Point origo(0,0);  
constexpr int z = origo.x;  
constexpr Point a[] = { Point(0,0), Point(1,1), Point(2,2) };  
constexpr int x = a[1].x;    // x becomes 1
```

- Note that the constructor can still be used in the usual way with non-constant parameters too



constexpr and const

Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

- Please note that `constexpr` is not a general purpose replacement for `const` (or vice versa)
 - `const`'s primary function
 - ▷ is to express the idea that an object is not modified through an interface (even though the object may very well be modified through other interfaces)
 - ▷ It just so happens that declaring an object `const` provides excellent optimization opportunities for the compiler
 - ▷ In particular, if an object is declared `const` and its address is not taken, a compiler is often able to evaluate its initializer at compile time (though that's not guaranteed) and keep that object in its tables rather than emitting it into the generated code
 - `constexpr`'s primary function
 - ▷ is to extend the range of what can be computed at compile time, making such computation type safe and also usable in compile-time contexts (such as to initialize enumerator or integral template parameters)
 - ▷ Objects declared `constexpr` have their initializer evaluated at compile time
 - ▷ they are basically values kept in the compiler's tables and only emitted into the generated code if needed



constexpr and const

- `constexpr` needs compile-time constant for initialization whereas `const` treats the initialized value as constant in run-time

```
#include <iostream>

constexpr int m = 100;           // Okay: m is 100: compile-time constant
                                // const will also work

void f(int n) {
    constexpr int c1 = m + 1;    // Okay: c1 is 101: compile-time constant

    // constexpr int c2 = n + 1; // Error: n is not compile-time constant
    const int c2 = n + 1;        // Okay: but value of c2 cannot be changed

    // constexpr int c3 = c2 + 1; // Error: c2 is not compile-time constant
    const int c3 = c2 + 1;       // Okay: but value of c3 cannot be changed

    std::cout << c1 << ' ' << c2 << ' ' << c3 << std::endl; // 101 11 12
}

int main() { f(10); }
```



Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

- `static` specifies the lifetime of the variable
- A `static constexpr` variable has to be set at compilation, because its lifetime is the the whole program
- Without the `static` keyword, the compiler is not bound to set the value at compilation, and could decide to set it later
- The most powerful thing about constant expressions, is that they enable us to do meta-programming without resorting to templates. So we can write a compile time function that computes factorial in a straightforward way:

```
constexpr unsigned int factorial(unsigned int n) {  
    return (n <= 1) ? 1 : (n * factorial(n - 1));  
}
```

```
static constexpr auto magic_value = factorial(5);
```



- `constexpr` cannot be used for all functions. For example:

```
constexpr int add_vectors_size(const vector<int>& a, const vector<int>& b)
{ return a.size() + b.size(); } // a.size() is not compile time constant
```

gives compilation error on `a.size()` as size of a vector is not compile time constant

- However, the following works fine:

```
#include <iostream>
#include <array> // Fixed size array
using namespace std;

template<size_t N1, size_t N2>
constexpr int add_arrays_size(const array<int, N1>& a, const array<int, N2>& b)
{ return a.size() + b.size(); } // a.size() is compile time constant

int main() { array<int, 10> p; array<int, 20> q;
    static constexpr auto n = add_arrays_size(p, q);
    cout << n << endl; // 30
}
```



constexpr (C++14)

Post-Recording

Module M48

Partha Pratim
Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

- In **C++11**, to make a function **constexpr** can mean rewriting it. Consider:

```
constexpr int my_charcmp(char c1, char c2) { return (c1==c2)? 0: (c1<c2)? -1: 1; }
```
- That is useful for characters. Can we extend it to strings? That would require iteration over the characters of the string, which **C++11** did not allow in **constexpr** functions, so the **C++11** version that supports strings would have to be recursive
- **C++14** allows more things inside the body of **constexpr** functions, notably:
 - local variables (not **static** or **thread_local**, and no *uninitialized variables*)
 - mutating objects whose lifetime began with the constant expression evaluation
 - **if**, **switch**, **for**, **while**, **do-while** (not **goto**)
- So in **C++14**, the above function generalized to strings can use a normal loop directly:

```
constexpr int my_strcmp(const char* str1, const char* str2) { int i = 0;  
    for( ; str1[i] && str2[i] && str1[i] == str2[i]; ++i) { }  
    if(str1[i] == str2[i]) return 0;  
    if(str1[i] < str2[i]) return -1;  
    return 1;  
}
```
- **C++14** also removes the **C++11** rule that **constexpr** member functions are implicitly **const**



noexcept: To prevent Exception Propagation

Module M48

Partha Pratim
Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

noexcept: To prevent Exception Propagation

Sources:

- [noexcept to prevent exception propagation, isocpp.org](http://isocpp.org)
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses



noexcept

Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

- If a function cannot throw an exception or if the program is not written to handle exceptions thrown by a function, that function can be declared **noexcept**:

```
extern "C" double sqrt(double) noexcept; // will never throw
```

```
// Not prepared to handle memory exhaustion
```

```
vector<double> my_computation(const vector<double>& v) noexcept {  
    vector<double> res(v.size()); // might throw  
    for(int i; i<v.size(); ++i) res[i] = sqrt(v[i]);  
    return res;  
}
```

- If a function declared **noexcept** throws (so that the exception tries to escape the **noexcept** function)
 - the program is terminated by a call to **std::terminate()**
 - the call of **terminate()** cannot rely on objects being in well-defined states, that is, there is
 - ▷ no guarantee that destructors have been invoked
 - ▷ no guaranteed stack unwinding, and
 - ▷ no possibility for resuming the program as if no problem had been encountered
 - This is deliberate and makes **noexcept** a simple, crude, and very efficient mechanism
 - ▷ much more efficient than the old dynamic **throw()** exception specification mechanism



noexcept

Module M48

Partha Pratim Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

- It is possible to make a function *conditionally noexcept*. For example, an algorithm can be specified to be *noexcept* iff the operations it uses on a template argument are *noexcept*:

```
template<class T>
// can throw if f(v.at(0)) can
void do_f(vector<T>& v) noexcept(noexcept(f(v.at(0)))) {
    for(int i; i<v.size(); ++i)
        v.at(i) = f(v.at(i));
}
```

- Here, we use *noexcept* as an operator:
 - noexcept(f(v.at(0)))* is true if *f(v.at(0))* cannot throw, that is,
 - if the *f()* and *at()* used are *noexcept*
- The *noexcept()* operator is a
 - constant expression* and
 - does not evaluate its operand*



noexcept

Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

- The general form of a `noexcept` declaration is
 - `noexcept(expression)`
 - *plain noexcept* is simply a shorthand for `noexcept(true)`
 - All declarations of a function must have compatible `noexcept` specifications
- A destructor should not throw
 - a generated destructor is implicitly `noexcept` (independently of what code is in its body) if all of the members of its class have `noexcept` destructors (which they too will have by default)
- It is typically a bad idea to have a *move operation throw*
 - declare those `noexcept` wherever possible
 - A *generated copy or move operation* is implicitly `noexcept` if all of the copy or move operations it uses on members of its class have `noexcept` destructors
- `noexcept` is *widely and systematically* used in the standard library to *improve performance and clarify requirements*



nullptr: null Pointer Literal

Module M48

Partha Pratim
Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

nullptr: null Pointer Literal

Sources:

- [nullptr – a null pointer literal](#), isocpp.org
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses
- [NULL](#), cppreference.com (C)
- [NULL](#), cppreference.com (C++)

Programming in Modern C++

Partha Pratim Das

M48.18



nullptr

Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

- `nullptr` is a literal denoting the null pointer
 - Literal of type `std::nullptr_t` in `<cstdint>`
 - Convertible to any pointer type and to `bool`, but nothing else
 - It is not an integer and cannot be used as an integral value
- `nullptr` is provided to replace the macro `NULL`
 - C Implementations (`<stddef.h>`, and others)

```
#define NULL 0 // C++ compatible
```

```
#define NULL (10*2 - 20) // C++ incompatible
```

```
#define NULL ((void*)0) // C++ incompatible
```
 - C++ Implementations (`<cstdint>`, and others)

```
#define NULL 0 // C++03. May be 0L in some compiler
```

```
#define NULL nullptr // C++11
```
- `NULL` or `0` causes confusion in following cases that `nullptr` can resolve:
 - **Function Overload Resolution**
 - **Forwarding Templates**



nullptr: Function Overload Resolution & Forwarding Template

Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

// Simple Examples

```
int* q = nullptr; // q is null
char* p = nullptr; // p is null
char* p1 = 0;      // 0 still works, p1 is null and p == p1
char* p2 = NULL;   // p2 is null
if (p) ...         // compiles but fails
if (p == p1) ...   // compiles and succeeds
if (q == p2) ...   // error: comparison between distinct pointer types int* and char*

void g(int);
g(nullptr);        // error: nullptr is not an int. cannot convert std::nullptr_t to int
int i = nullptr;   // error: nullptr is not an int. cannot convert std::nullptr_t to int

void f(int); void f(int*); // Function overload resolution
f(0);                    // call f(int)
f(nullptr);               // call f(int*)
f(NULL);                  // error: call of overloaded f(NULL) is ambiguous for f(int) and f(int*)

void h(int*);           // h(0) and h(nullptr) are okay
template<typename F, typename P> // Forwarding template
void logAndCall(F func, P param) {
    ... func(param); // make log entry ..., then invoke func on param
}
logAndCall(h, 0);        // error: P deduced as int, and h(int) invalid
logAndCall(h, NULL);     // error: P deduced as long int, and h(long int) invalid
logAndCall(h, nullptr);  // P deduced as std::nullptr_t, and h(std::nullptr_t) is okay
```

Programming in Modern C++



Inline namespaces: Efficient Version Management

Module M48

Partha Pratim
Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

Inline namespaces: Efficient Version Management

Sources:

- [Inline namespaces](#), isocpp.org
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses
- [Inline namespaces and usage of the "using" directive inside namespaces](#), geeksforgeeks.org, 2021



Inline namespaces

Module M48

Partha Pratim Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

- The `inline namespace` mechanism is intended to support library evolution by providing a mechanism that supports a form of versioning. Consider:

```
// file V99.h:
inline namespace V99 {
    void f(int);    // does something better than the V98 version
    void f(double); // new feature
    // ...
}

// file V98.h:
namespace V98 {
    void f(int);    // does something
    // ...
}

// file Mine.h:
namespace Mine {
    #include "V99.h"
    #include "V98.h"
}
```



Inline namespaces

Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

- We here have a namespace `Mine` with both the latest release (`V99`) and the previous one (`V98`). If we want to be specific, we can:

```
#include "Mine.h"
using namespace Mine;
// ...
V98::f(1); // old version
V99::f(1); // new version
f(1);      // default version
```

- The point is that the `inline` specifier makes the declarations from the nested namespace appear exactly as if they had been declared in the enclosing namespace.
- This is a very *static* and *implementer-oriented facility* in that the inline specifier has to be placed by the designer of the namespaces – thus making the choice for all users
 - It is not possible for a user of `Mine` to say:
 - ▷ *I want the default to be V98 rather than V99*



Inline namespaces:

namespace in C++03 vs. inline namespace in C++11

Module M48

Partha Pratim
Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

// C++03: nested namespaces

```
#include <iostream>
using namespace std;
```

```
namespace ns1 { int v1 = 2;
    namespace ns2 { int v2 = 3;
        namespace ns3 { int v3 = 5;
            }
        }
    }

int main() { // Fully qualified names must
    cout << ns1::v1 << ' ';
    cout << ns1::ns2::v2 << ' ';

    cout << ns1::ns2::ns3::v3 << endl;
}
2 3 5
```

- **Note:** If the outermost namespace (`ns1`) is `inline`, then the symbols within `ns1` are available in the global namespace. For example, `ns1::ns2::ns3::v3` can be accessed as `v3` in `main()` besides as `ns1::ns2::v3` and `ns1::v3`. This property is used in **Version Control**

// C++11: inline namespaces

```
#include <iostream>
using namespace std;
```

```
// inline namespace ns1; for global access
namespace ns1 { int v1 = 2;
    inline namespace ns2 { int v2 = 3;
        inline namespace ns3 { int v3 = 5;
            }
        }
    }

int main() { // Qualified by enclosing namespace
    cout << ns1::v1 << ' ';
    cout << ns1::v2 << ' ';
    cout << ns1::ns2::v3 << ' ';
    cout << ns1::v3 << endl;
}
2 3 5 5
```



Inline namespaces:

inline namespace effects by using namespace in C++03

Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

```
// C++03: nested namespaces
#include <iostream>
using namespace std;

namespace ns1 { int v1 = 2;
    namespace ns2 { int v2 = 3;
        namespace ns3 { int v3 = 5;
            }
        }
    }

int main() { // Fully qualified names must
    cout << ns1::v1 << ' ';
    cout << ns1::ns2::v2 << ' ';

    cout << ns1::ns2::ns3::v3 << endl;
}
```

- **Note:** With `using namespace ns1` before `main()` the symbols within `ns1` will be in the global namespace. Like, `ns1::ns2::ns3::v3` can be accessed as `ns1::ns2::v3`, `ns1::v3`, and `v3`
- However, `using namespace ns1` belongs to the *application space*. Hence, **the choice of putting it belongs to the user and default version cannot be forced**. `inline namespace` addresses this *default enforcement* for **Version Control**

```
// C++03: inline namespace effect by using
#include <iostream>
using namespace std;

namespace ns1 { int v1 = 2;
    namespace ns2 { int v2 = 3;
        namespace ns3 { int v3 = 5;
            }
        using namespace ns3;
    }
    using namespace ns2;
} // using namespace ns1; for global access

int main() { // Qualified by using namespaces
    cout << ns1::v1 << ' ';
    cout << ns1::v2 << ' ';
    cout << ns1::ns2::v3 << ' ';
    cout << ns1::v3 << endl;
}
```



static_assert: Compile-time Assertions

Module M48

Partha Pratim Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

static_assert: Compile-time Assertions

Sources:

- [static_assert: Compile-time Assertions](https://ericniebler.com/2014/07/14/static-assert/), [isocpp.org](https://ericniebler.com/2014/07/14/static-assert/)



- A static (compile time) assertion consists of a constant expression and a string literal:
`static_assert(expression, string);`
- The compiler evaluates the expression and writes the string as an error message if the expression is false (that is, if the assertion failed). For example: (More example in **Module 51**)

```
static_assert(sizeof(long)>=8, "64-bit code generation required for this library");  
struct S { X m1; Y m2; };  
static_assert(sizeof(S)==sizeof(X)+sizeof(Y), "unexpected padding in S");
```
- A `static_assert` can be useful to make assumptions about a program and its treatment by a compiler explicit. Note that since `static_assert` is evaluated at compile time, it cannot be used to check assumptions that depends on run-time values:

```
int f(int* p, int n) {  
    static_assert(p==0, "p is not null"); // error: static_assert expression  
                                           // not a constant expression  
    // ...  
}
```
- Instead, we should use a normal `assert(p==0 && "p is not null");` or test and throw an exception in case of failure



User-defined Literals: UDTs closer to Built-in Types

Module M48

Partha Pratim
Das

Objectives &
Outlines

`constexpr`

`noexcept`

`nullptr`

Inline namespaces

`static_assert`

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

User-defined Literals: UDTs closer to Built-in Types

Sources:

- [User-defined literals](#), isocpp.org
- [User Defined Literals in C++](#), geeksforgeeks.org, 2018
- [User-defined literals](#), microsoft.com, 2021
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses



User-defined Literals

Module M48

Partha Pratim
Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

- C++ has always provided literals for a variety of built-in types:

```
123      // int
1.2      // double
1.2F     // float
'a'      // char
1ULL     // unsigned long long
0xD0     // hexadecimal unsigned
"as"     // string
```

- However, in C++03 there are no literals for user-defined types. This violates the principle that UDTs should be supported as well as built-in types are. Common requests include:

```
"Hi!"s           // std::string, not
                  // "zero-terminated array of char"
1.2i             // imaginary
123.4567891234df // decimal floating point (IBM)
101010111000101b // binary
123s            // seconds
123.56km        // not miles! (units)
1234567890123456789012345678901234567890x // extended-precision
```




User-defined Literals: Literal Operators

Module M48

Partha Pratim
Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

- C++11 supports *user-defined literals* through the notion of *literal operators* that *map literals with a given suffix into a desired type*. For example:

```
constexpr complex<double> operator "" _i(long double d) { // imaginary literal
    return complex<double>{ 0.0, static_cast<double>(d) }; // complex is a literal type
} // Note the use of constexpr to enable compile-time evaluation
```
- Literal operator has the syntax: `<ReturnType> operator "" <Suffix> (<Parameters>)`:
 - **ReturnType** can be anything including `void`
 - **Suffix** must start with an underscore (`_`). Only the Standard Library is allowed to define literals without the underscore. Suffixes will tend to be short (like `_s` for `string`, `_i` for `imaginary`, `_m` for `meter`, and `_x` for `extended`), so different uses could easily clash. Use namespaces to prevent clashes:
 - **Parameters** can be any one of four kinds of literals
 - ▷ *Integer literal*: `unsigned long long int`
 - ▷ *Floating-point literal*: `long double`
 - ▷ *String literal*: `(const char*, size_t)`, `(const wchar_t*, size_t)`, `(const char16_t*, size_t)`, `(const char32_t*, size_t)`, or `(char const*)`,
 - ▷ *Character literal*: `char`, `wchar_t`, `char16_t`, or `char32_t`



User-defined Literals: `std::string` Literals

Module M48

Partha Pratim
Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

```
#include <iostream>
#include <string>
using namespace std;

std::string operator"" _s(const char* p, size_t n) { // std::string literal
    return string(p, n); // requires free store allocation
}

template<class T> void f(const T& a) {
    cout << a << endl;
}

int main() {
    f("Hello");           // pass pointer to char* => const char (&)[6]
    f("Hello"_s);         // pass (5-character) std::string object
    f("Hello\n"_s);       // pass (6-character) std::string object
}
```



User-defined Literals: `std::complex` Literals

Module M48

Partha Pratim
Das

Objectives &
Outlines

`constexpr`

`noexcept`

`nullptr`

Inline namespaces

`static_assert`

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

```
#include <iostream>
#include <complex>
using namespace std;

// Note the use of constexpr to enable compile-time evaluation
constexpr complex<double> operator "" _i(long double d) { // imaginary literal
    return complex<double>{ 0.0, static_cast<double>(d) }; // complex is a literal type
}

int main() {
    auto z = 3.0 + 4.0_i;           // complex(3.0, 4.0)
    auto y = 2.3 + 5.0_i;          // complex(2.3, 5.0)
    cout << "z + y = " << z+y << endl; // z + y = (5.3,9)
    cout << "z * y = " << z*y << endl; // z * y = (-13.1,24.2)
    cout << "abs(z) = " << abs(z) << endl; // abs(z) = 5
}
```



User-defined Literals: Metric Weight Literals

Module M48

Partha Pratim
Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

```
#include<iostream>
#include<iomanip>
using namespace std;

// user defined literals: kg, g, and mg
long double operator"" _kg(long double x) { // KiloGram: to gram
    return x * 1000;
}
long double operator"" _g(long double x) { // Gram
    return x;
}
long double operator"" _mg(long double x) { // MiliGram: to gram
    return x / 1000;
}

int main() {
    long double weight = 3.6_kg;
    cout << weight << endl; // 3600
    cout << setprecision(8) << (weight + 2.3_mg) << endl; // 3600.0023
    cout << (32.3_kg / 2.0_g) << endl; // 16150
    cout << (32.3_mg * 2.0) << endl; // 0.0646
}
```

Programming in Modern C++



User-defined Literals: Date Literals

Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

```
#include <iostream>
#include <cstring>
using namespace std;
// User-defined Date class
class Date { int date, month, year;
public: Date(int d = 1, int m = 1, int y = 0): date(d), month(m), year(y) { }
    friend ostream& operator<<(ostream& os, const Date& d) {
        os << d.date << "/" << d.month << "/" << d.year; return os;
    }
};
// Literal operator for Date
Date operator "" _ad(const char* s, size_t) { // representation of date as "dd/mm/yyyy" format
    // parsing s into dd, mm, yyyy as int
    char *str = strdup(s); // copy needed as s is const char* - strtok cannot work on s
    char *date_str = strtok(str, "/"); int date = atoi(date_str);
    date_str = strtok(NULL, "/"); int month = atoi(date_str);
    date_str = strtok(NULL, "/"); int year = atoi(date_str);
    free(str);

    return Date{ date, month, year }; // Date is a literal type
}
int main() {
    auto myDate = "08/02/2022"_ad; // Date object created from literal
    cout << myDate << endl;
}
```

Programming in Modern C++



User-defined Literals

Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

- The basic (implementation) idea is
 - After parsing what could be a *literal*, the compiler always checks for a *suffix*
 - The user-defined literal mechanism simply *allows the user to specify a new suffix* and what is to be done with the literal before it
 - *It is not possible to redefine the meaning of a built-in literal suffix or augment the syntax of literals*
 - A literal operator can request to get its (preceding) literal passed
 - ▷ as *cooked* (with the value it would have had if the new suffix had not been defined) or
 - ▷ as *uncooked* (as a string) by simply requesting a single `const char*` argument:

```
Bignum operator"" x(const char* p) {  
    return Bignum(p);  
}  
  
void f(Bignum);  
f(1234567890123456789012345678901234567890x);
```

Here the C-style string "1234567890123456789012345678901234567890" is passed to `operator"" x()`. Note that we did not explicitly put those digits into a string



Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

- C++11 added user-defined literals, but did not use them in the standard library
- Now some very useful and popular ones work in C++14 for `std::` types:

```
auto a_string = "hello there"s;    // type std::string
auto a_minute = 60s;               // type std::chrono::duration = 60 seconds
auto a_day     = 24h;               // type std::chrono::duration = 24 hours
```
- Note `s` means *string* when used on a *string literal*, and *seconds* when used on an *integer literal*, without ambiguity



Digit Separators and Binary Literals

Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

Digit Separators and Binary Literals

Sources:

- [Digit Separators](https://ericniebler.com/2014/06/01/digit-separators/), isocpp.org
- [Binary Literals](https://ericniebler.com/2014/06/01/digit-separators/), isocpp.org



- **Digit Separator**

- In **C++14**, the single-quote character ' can be used anywhere within a numeric literal for aesthetic readability. It does not affect the numeric value

```
auto million = 1'000'000;  
auto pi = 3.14159'26535'89793;
```

- **Binary Literals**

- **C++14** supports binary literals:

```
auto a1 = 42;           // ... decimal  
auto a2 = 0x2A;         // ... hexadecimal  
auto a3 = 0b101010;     // ... binary
```

- This works well in combination with the new ' digit separators, for example, to separate nybbles or bytes:

```
auto a = 0b100'0001;    // ASCII 'A'
```



Raw String Literals

Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

Raw String Literals

Sources:

- [Raw string literals](#), isocpp.org
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses



Raw String Literals

Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

- String literals where *special* characters are *not special*:

- For example, escaped characters and double quotes:

```
std::string noNewlines(R"(\n\n)");  
std::string cmd(R"(ls /home/docs | grep ".pdf")");
```

- For example, newlines:

```
std::string withNewlines(R"  
                        Line 1 of the string...  
                        Line 2...  
                        Line 3)");
```

- Rawness* may be added to any string encoding:

```
LR"(Raw Wide string literal \t (without a tab))"  
u8R"(Raw UTF-8 string literal \n (without a newline))"  
uR"(Raw UTF-16 string literal \\ (with two backslashes))"  
UR"(Raw UTF-32 string literal 2620 (w/o a skull & crossbones))"
```

- Raw text delimiters may be customized:

- Useful when `)` is in raw text, for example, in regular expressions:

```
std::regex re1(R"! ("operator"|"operator->")!"); // "operator()"|"operator->"  
std::regex re2(R"xyzzzy("\([A-Za-z_] \w *\)")xyzzzy"); // "(identifier)" \
```



Unicode Support

Module M48

Partha Pratim
Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

Unicode Support

Sources:

- [Unicode Support in the Standard Library](#), isocpp.org, 2013
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses
- [A Modern C++ and Unicode primer](#), cpptutor, 2021



Module M48

Partha Pratim
Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

- For unicode support, C++11 adds two new character types:

```
char16_t           // 16-bit character (if available); akin to uint_least16_t
char32_t           // 32-bit character (if available); akin to uint_least32_t
```

- Literals of these types prefixed with u/U, are UCS-encoded:

```
u'x'               // 'x' as a char16_t using UCS-2
U'x'               // 'x' as a char32_t using UCS-4/UTF-32
```

- C++98 character types still exist, of course:

```
'x'                // 'x' as a char
L'x'               // 'x' as a wchar_t
```

- There are corresponding string literals:

```
u"UTF-16 string literal" // => char16_ts in UTF-16
U"UTF-32 string literal"  // => char32_ts in UTF-32/UCS-4
"Ordinary/narrow string literal" // "ordinary/narrow" => chars
L"Wide string literal"    // "wide" => wchar_ts
```

- UTF-8 string literals are also supported:

```
u8"UTF-8 string literal" // => chars in UTF-8
```



- Code points can be specified via `\unnnn` and `\Unnnnnnnnn`:

```
u8"G clef: \U0001D11E"           // 🎵
u"Thai character Khomut: \u0E5B" // คอ
U"Skull and crossbones: \u2620"  // ☠
```

- There are `std::basic_string` typedefs for all character types:

```
std::string s1;    // std::basic_string<char>
std::wstring s2;   // std::basic_string<wchar_t>
std::u16string s3; // std::basic_string<char16_t>
std::u32string s4; // std::basic_string<char32_t>
```

- C++98 guarantees only two `codecvt` facets for conversions among encodings:

- `char` $\leftrightarrow$ `char` (`std::codecvt<char, char, std::mbstate_t>`)
 - ▷ “Degenerate” – no conversion performed
- `wchar_t` $\leftrightarrow$ `char` (`std::codecvt<wchar_t, char, std::mbstate_t>`)

- C++11 adds more facets for conversions among encodings:

- `UTF-16` $\leftrightarrow$ `UTF-8` (`std::codecvt<char16_t, char, std::mbstate_t>`)
- `UTF-32` $\leftrightarrow$ `UTF-8` (`std::codecvt<char32_t, char, std::mbstate_t>`)
- `UTF-8` $\leftrightarrow$ `UCS-2`, `UTF-8` $\leftrightarrow$ `UCS-4` (`std::codecvt_utf8`)
- `UTF-16` $\leftrightarrow$ `UCS-2`, `UTF-16` $\leftrightarrow$ `UCS-4` (`std::codecvt_utf16`)
- `UTF-8` $\leftrightarrow$ `UTF-16` (`std::codecvt_utf8_utf16`)

▷ Behaves like `std::codecvt<char16_t, char, std::mbstate_t>`



Memory Alignment

Module M48

Partha Pratim Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

Memory Alignment

Sources:

- [Alignment](#), isocpp.org
- [alignof operator](#), cppreference
- [alignas specifier](#), cppreference



Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

- C++11 introduces keywords, `alignof` and `alignas`, to support control of memory alignment
- The `alignof` keyword can get a platform-dependent value of type `std::size_t` to query the alignment of the platform
- In addition, `alignas` customizes the alignment of the structure

```
#include <iostream>
#include <cstdint> // max_align_t
struct Storage { // alignof = 8, sizeof = 24
    char a; int b; double c; long long d;
};
struct alignas(std::max_align_t) AlignasStorage { // alignof = 16, sizeof = 32
    char a; int b; double c; long long d;
};
int main() {
    std::cout << alignof(Storage) << ' ' << sizeof(Storage) << std::endl;
    std::cout << alignof(AlignasStorage) << ' ' << sizeof(AlignasStorage) << std::endl;
}
```

- `std::max_align_t` requires the same alignment for each scalar type, so it has almost no difference in maximum scalars
- The result on most platforms is `long double`, so the alignment requirement for `AlignasStorage` we get here is 8 or 16



Attributes

Module M48

Partha Pratim
Das

Objectives &
Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined
Literals

Digit Separators
/ Binary Literals

Raw String
Literals

Unicode Support

Memory
Alignment

Attributes

Module Summary

Attributes

Sources:

- [Attributes](#), isocpp.org



- **Attributes** is a new standard syntax for adding optional and/or vendor specific information into source code (for example, `__attribute__`, `__declspec`, and `#pragma`)
- **C++11** attributes differ from existing syntaxes by being applicable essentially everywhere in code and always relating to the immediately preceding syntactic entity. For example:

```
void f [ [ noreturn ] ] () { // attribute is placed in [ [ ... ] ]. f() never returns
    throw "error"; // OK
}

struct foo* f [ [ carries_dependency ] ] (int i); // hint to optimizer
int* g(int* x, int* y [ [ carries_dependency ] ] );
```
- `[ [ noreturn ] ]` and `[ [ carries_dependency ] ]` are the two attributes defined in the standard
- The use of attributes should only control things that do not affect the meaning of a program but might help detect errors (`[ [ noreturn ] ]`) or help optimizers (`[ [ carries_dependency ] ]`)
- One planned use for attributes is improved support for OpenMP. For example:

```
for [ [ omp::parallel() ] ] (int i=0; i<v.size(); ++i) { } // ... This may be risky as
// semantics of a parallel loop are decidedly not the same as a sequential loop
```
- In **C++14**, the `deprecated` attribute allows marking an entity deprecated, which is still legal to use but puts users on notice that use is discouraged and may cause a compilation warning
- Applicable to the declaration of a class, typedef-name, variable, non-static data member, function, enumeration, or template specialization



Module Summary

Module M48

Partha Pratim Das

Objectives & Outlines

constexpr

noexcept

nullptr

Inline namespaces

static_assert

User-defined Literals

Digit Separators / Binary Literals

Raw String Literals

Unicode Support

Memory Alignment

Attributes

Module Summary

- Introduced following **C++11** general features:
 - constexpr (+ **C++14**)
 - noexcept
 - nullptr
 - Inline namespace
 - static_assert
 - User-defined Literals (+ **C++14**)
 - Digit Separators and Binary Literals (+ **C++14**)
 - Raw String Literals
 - Unicode Support
 - Memory Alignment
 - Attributes (+ **C++14**)



Module M49

Partha Pratim
Das

Objectives &
Outlines

Copying vs.
Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow
Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue
Vector

Implementing
Move Semantics

Module Summary

Programming in Modern C++

Module M49: C++11 and beyond: General Features: Part 4: Rvalue and Move/1

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs.

Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow

Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue

Vector

Implementing

Move Semantics

Module Summary

- Introduced following **C++11** general features:
 - constexpr (+ **C++14**)
 - noexcept
 - nullptr
 - Inline namespace
 - static_assert
 - User-defined Literals (+ **C++14**)
 - Digit Separators and Binary Literals (+ **C++14**)
 - Raw String Literals
 - Unicode Support
 - Memory Alignment
 - Attributes (+ **C++14**)



Module Objectives

Module M49

Partha Pratim
Das

Objectives & Outlines

Copying vs.
Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow
Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue
Vector

Implementing
Move Semantics

Module Summary

- Understanding the difference between Copying and Moving
- Understanding the difference between Lvalue and Rvalue
- Exploiting the advantages of Move in C++ using
 - Rvalue Reference
 - Move Semantics
 - Copy / Move Constructor / Assignment
 - Implementation of Move Semantics



Module Outline

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue Vector

Implementing Move Semantics

Module Summary

- 1 Copying vs. Moving
 - Return Value
 - Append Full Vector
 - Swap
 - Deep vs. Shallow Copy
 - Performance Test
- 2 Rvalue References and Move Semantics
 - Rvalue References
 - Copy vs. Move
 - Lvalue vs. Rvalue
 - Vector
- 3 Implementing Move Semantics
- 4 Module Summary



Copying vs. Moving

Module M49

Partha Pratim
Das

Objectives &
Outlines

Copying vs.
Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow
Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue
Vector

Implementing
Move Semantics

Module Summary

Copying vs. Moving

Sources:

- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses
- [Scott Meyers on C++](#)



Copying vs. Moving

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

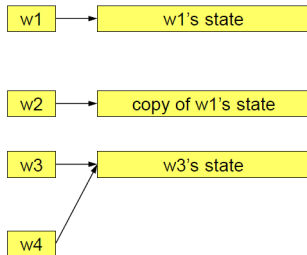
Lvalue vs. Rvalue Vector

Implementing Move Semantics

Module Summary

- C++ has always supported copying object state:
 - *Copy* constructors, *Copy* assignment operators
- C++11 adds support for requests to *Move* object state:

```
Widget w1;  
...  
// copy w1's state to w2  
Widget w2(w1);  
Widget w3;  
...  
// move w3's state to w4  
Widget w4(std::move(w3));
```



- **Note:** *w3* continues to exist in a valid state after creation of *w4*



Copying vs. Moving: Return Value

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

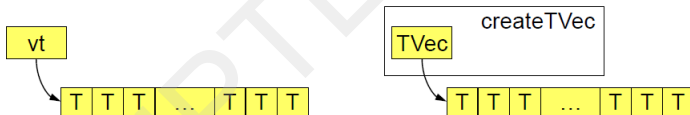
Lvalue vs. Rvalue Vector

Implementing Move Semantics

Module Summary

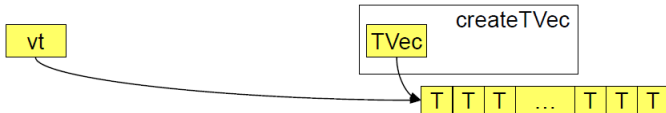
- C++ at times performs *extra copy*, while *temporary objects* are prime candidates for *move*:

```
typedef std::vector<T> TVec;
TVec createTVec(); // factory function
TVec vt;
...
vt = createTVec(); // in C++03, copy return value to vt, then destroy return value
```



- *Moving* values would be cheaper and C++11 generally turns such copy operations into moves:

```
TVec vt;
...
vt = createTVec(); // implicit move request in C++11. move data in return value object to vt
// then destroy return value object
```





Copying vs. Moving: Append a Full Vector

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue

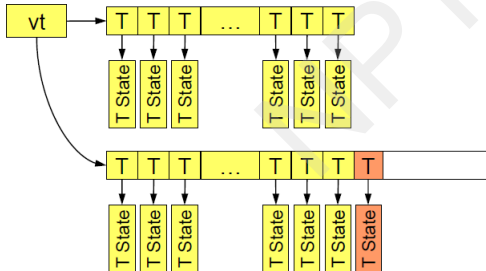
Vector

Implementing Move Semantics

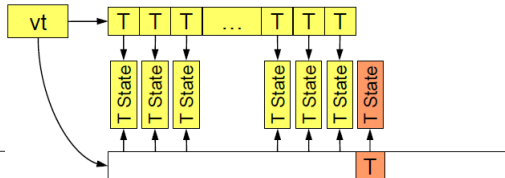
Module Summary

- Appending to a full vector causes much *copying before the append*. Moving would be efficient:
assume **vt** lacks unused capacity

```
std::vector<T> vt;  
...  
vt.push_back(T object);
```



```
std::vector<T> vt;  
...  
vt.push_back(T object);
```



- **vector** and **deque** operations like **insert**, **emplace**, **resize**, **erase**, etc. would benefit too



Copying vs. Moving: Swap

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue Vector

Implementing Move Semantics

Module Summary

- Consider swapping two values:

By Copy

```
template<typename T> void swap(T& a, T& b) { // std::swap impl. by copy
void swap(T& a, T& b) {
    T tmp(a);           // copy a to tmp (=> 2 copies of a)
    a = b;              // copy b to a (=> 2 copies of b)
    b = tmp;            // copy tmp to b (=> 2 copies of tmp)
}                       // destroy tmp
```

By Move

```
template<typename T> void swap(T& a, T& b) { // std::swap impl. by move
    T tmp(std::move(a)); // move a's data to tmp
    a = std::move(b);    // move b's data to a
    b = std::move(tmp);  // move tmp's data to b
}                       // destroy (eviscerated) tmp
```



Copying vs. Moving: Deep vs. Shallow Copy

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

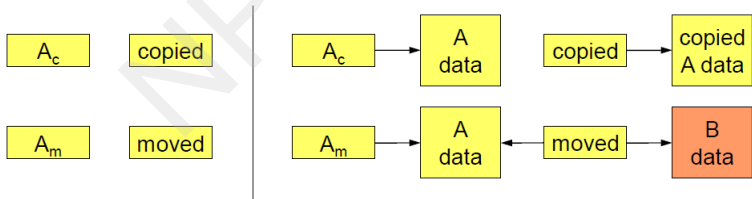
Lvalue vs. Rvalue

Vector

Implementing Move Semantics

Module Summary

- Moving most important when:
 - *Object has data in separate memory (for example, on free store).*
 - *Copying is deep*
- Moving copies only object memory
 - Copying copies object memory + **separate memory**
- Consider copying/moving A to B:



- **Moving never slower than copying, and often faster**



Simple Performance Test

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue

Vector

Implementing Move Semantics

Module Summary

- Given

```
const std::string stringValue("This string has 29 characters");
```

```
class Widget { std::string s;  
public:  
    Widget(): s(stringValue) { }  
    ...  
};
```

- Consider this `push_back`-based loop:

```
std::vector<Widget> vw;  
Widget w;  
for (std::size_t i = 0; i < n; ++i) { // append n copies of w to vw  
    vw.push_back(w);  
}
```



Performance Data

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow

Copy

Performance Test

Rvalue & Move

Rvalue References

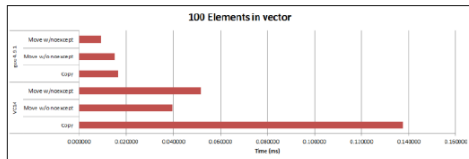
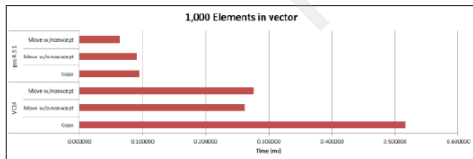
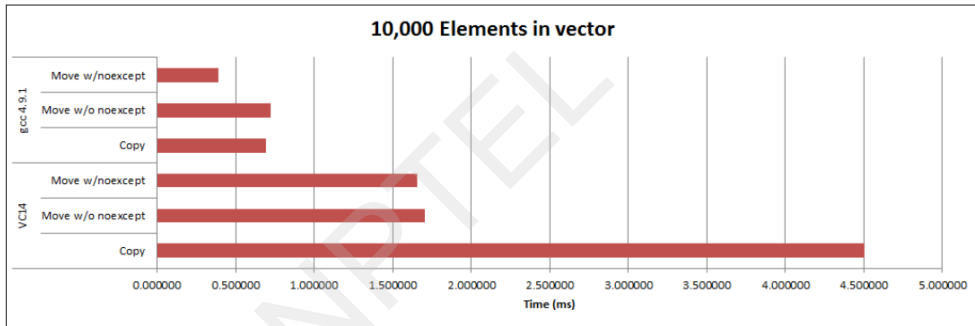
Copy vs. Move

Lvalue vs. Rvalue

Vector

Implementing Move Semantics

Module Summary





Copying vs. Moving

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue Vector

Implementing Move Semantics

Module Summary

- Lets C++ recognize move opportunities and take advantage of them.
 - **How recognize them?**
 - **How take advantage of them?**
- **Moving a key new C++11 idea**
 - Usually an optimization of copying
- Most standard types in C++11 are *move-enabled*
 - They support move requests
 - For example, STL containers
- Some types are *move-only*:
 - Copying prohibited, but moving is allowed
 - For example, stream objects, `std::thread` objects, `std::unique_ptr`, etc.



Rvalue References and Move Semantics

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector
Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue
Vector

Implementing
Move Semantics

Module Summary

Sources:

- [Rvalue references and move semantics](http://isocpp.org), isocpp.org
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses
- Rvalue References
 - [C++ Rvalue References Explained](#)
 - [Lvalues and Rvalues](#), accu.org, 2004
 - [What are rvalues, lvalues, xvalues, glvalues, and prvalues?](#), stackoverflow.com, 2010
- Move Semantics
 - [What is move semantics?](#), stackoverflow.com, 2010
 - [M.3 — Move constructors and move assignment](#), learncpp.com, 2021
 - [Move Constructors and Move Assignment Operators \(C++\)](#), Microsoft, 2021
 - Understanding Move Semantics and Perfect Forwarding: [Part 1](#), [Part 2: Rvalue References and Move Semantics](#), [Part 3: Perfect Forwarding](#), Drew Campbell, 2018

Rvalue References and Move Semantics



Lvalues and Rvalues

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue Vector

Implementing Move Semantics

Module Summary

- **Lvalues** are generally things we can take the address of:
 - In C, Expressions on *left-hand-side (LHS)* of an assignment
 - Named objects - variables
 - Legal to apply address of (&) operator
 - **Lvalue** references
- **Rvalues** are generally things we cannot take the address of:
 - In C, Expressions on *right-hand-side (RHS)* of an assignment
 - Typically unnamed temporary objects - expressions, return values from functions, etc.
 - **Rvalue** references
- Examples:

```
int x, *pInt;           // x, pInt, *pInt are lvalues
std::size_t f(std::string str); // f and str are lvalues, f's return is rvalue
f("Hello");           // temp string("Hello") created for call is rvalue
std::vector<int> vi;    // vi is lvalue
...
vi[5] = 0;             // vi[5] is lvalue
```

 - Recall that `vector<T>::operator[]` returns **T&**



Moving and Lvalues

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue Vector

Implementing Move Semantics

Module Summary

- *Value movement generally not safe when the source is an **lvalue** object*

- That continues to exist, may be referred to later:

```
TVec vt1;
...
TVec vt2(vt1);           // it is expected that vt1 be copied to vt2, not moved!
...use vt1...           // value of vt1 here should be same as above
```

- *Value movement is safe when the source is an **rvalue** object*

- Temp's usually *go away at statement's end*. No way to tell if their *value has been modified*

```
TVec createTVec();           // as before
TVec vt1;
vt1 = createTVec();          // rvalue source: move okay
auto vt2 = createTVec();     // rvalue source: move okay
vt1 = vt2;                   // lvalue source: copy needed
auto vt3(vt2);               // lvalue source: copy needed
std::size_t f(std::string str); // as before
f("Hello");                  // rvalue (temp) source: move okay
std::string s("C++11");
f(s);                         // lvalue source: copy needed
```



Rvalue References

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow

Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue

Vector

Implementing

Move Semantics

Module Summary

- C++11 introduces **rvalue references**
 - Syntax: **T&&**
 - **Normal references** now known as **lvalue references**
 - ▷ **Rvalue references** behave similarly to **Lvalue references**
 - Must be initialized, cannot be rebound, etc.
- **Rvalue references identify objects that may be moved from**
- Reference Binding Rules
 - Important for overloading resolution
 - As always:
 - ▷ **Lvalues** may bind to **lvalue references**
 - ▷ **Rvalues** may bind to **lvalue references to const**
 - In addition:
 - ▷ **Rvalues** may bind to **rvalue references to non-const**
 - ▷ **Lvalues** may *not* bind to **rvalue references**
 - Otherwise lvalues could be accidentally modified



Rvalue References

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue Vector

Implementing Move Semantics

Module Summary

- Examples:

```
void f1(const TVec&);           // takes const lvalue ref
TVec vt;
f1(vt);                         // fine (as always)
f1(createTVec());              // fine (as always)
void f2(const TVec&);           // #1: takes const lvalue ref
void f2(TVec&&);                // #2: takes non-const rvalue ref
f2(vt);                         // lvalue => #1
f2(createTVec());              // both viable, non-const rvalue => #2
void f3(const TVec&&);           // #1: takes const rvalue ref
void f3(TVec&&);                // #2: takes non-const rvalue ref
f3(vt);                         // error! lvalue
f3(createTVec());              // both viable, non-const rvalue => #2
```



Rvalue References and const

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue

Vector

Implementing Move Semantics

Module Summary

- C++ remains const-correct:
 - `const lvalues / rvalues` bind only to `references-to-const`
- But `rvalue-references-to-const` are essentially useless
 - Rvalue references designed for two specific problems:
 - ▷ **Move semantics**
 - ▷ **Perfect forwarding**
 - C++11 language rules carefully crafted for these needs
 - ▷ `Rvalue-refs-to-const` not considered in these rules
 - `const T&&`s are legal, but not designed to be useful
 - ▷ Uses already emerging :-)
- Implications:
 - Do not declare `const T&&` parameters
 - Not possible to move from them, anyway
 - Hence this rarely makes sense:



Distinguishing Copying from Moving

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue Vector

Implementing Move Semantics

Module Summary

- Overloading exposes move-instead-of-copy opportunities:

```
class Widget { public:
    Widget(const Widget&);           // copy constructor
    Widget(Widget&&) noexcept;       // move constructor
    Widget& operator=(const Widget&); // copy assignment op
    Widget& operator=(Widget&&) noexcept; // move assignment op
    ...
};
Widget createWidget(); // factory function
Widget w1;
Widget w2 = w1;        // lvalue src => copy required
w2 = createWidget();   // rvalue src => move okay
w1 = w2;               // lvalue src => copy required
```

- Move operations need not be `noexcept`, but it is preferable
 - Moves should be fast, and `noexcept` => more optimizable
 - Some contexts require `noexcept` moves (for example, `std::vector::push_back`)
 - Move operations often have natural `noexcept` implementations
- We declare move operations `noexcept` by default



Copy vs. Move : Lvalue vs. Rvalue

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue

Vector

Implementing Move Semantics

Module Summary

```
class A { public: A() { std::cout << "Defa Ctor" << endl; } // Defa Constructor
    A(const A&) { std::cout << "Copy Ctor" << endl; } // Copy Constructor
    A(A&&) noexcept { std::cout << "Move Ctor" << endl; } // Move Constructor
    A& operator=(const A&) { cout << "Copy =" << endl; return *this; } // Copy =
    A& operator=(A&&) noexcept { cout << "Move =" << endl; return *this; } // Move =
    friend A operator+(const A& a, const A& b) { A t; return t; } // Temp. obj. ret.-by-value
};
```

		Only Copy		Copy & Move	
		Debug	Release	Debug	Release
A a;	// lvalue	Defa Ctor	Defa Ctor	Defa Ctor	Defa Ctor
A b = a;	// lvalue	Copy Ctor	Copy Ctor	Copy Ctor	Copy Ctor
A c = a + b;	// rvalue	Defa Ctor	Defa Ctor	Defa Ctor	Defa Ctor
// RVO in a + b for release build		Copy Ctor	// RVO	Move Ctor	// RVO
A d = std::move(a);	// rvalue	Copy Ctor	Copy Ctor	Move Ctor	Move Ctor
b = a;	// lvalue	Copy =	Copy =	Copy =	Copy =
c = a + b;	// rvalue	Defa Ctor	Defa Ctor	Defa Ctor	Defa Ctor
		Copy Ctor	// RVO	Move Ctor	// RVO
		Copy =	Copy =	Move =	Move =

- Return Value Optimization (**RVO**) eliminates the temp. obj. created to hold a function's return value
- **std::move(t)** produces a rvalue from **t** to indicate that the object **t** may be *moved from*



Copy vs. Move : Lvalue vs. Rvalue: Explanation

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector
Swap

Deep vs. Shallow
Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue
Vector

Implementing
Move Semantics

Module Summary

```

class A { public: A() { std::cout << "Defa Ctor" << endl; }           // Defa Constructor
        A(const A&) { std::cout << "Copy Ctor" << endl; }           // Copy Constructor
        A(A&&) noexcept { std::cout << "Move Ctor" << endl; }       // Move Constructor
        A& operator=(const A&) { cout << "Copy =" << endl; return *this; } // Copy =
        A& operator=(A&&) noexcept { cout << "Move =" << endl; return *this; } // Move =
        friend A operator+(const A& a, const A& b) { A t; return t; } // Temp. obj. ret.-by-value
};

```

- $A \ a;$ $\Rightarrow$ a is an **lvalue** and is **default constructed**
- $A \ b = a;$ $\Rightarrow$ a is an **lvalue** and hence, b is **copy constructed**
- $A \ c = a + b;$ $\Rightarrow$ **operator+(a, b)** **default constructs** t , computes the result of $a + b$ in t (not shown) and then **returns t by value**. Hence $a + b$ is an **rvalue** and c is **move constructed** (if available, else **copy constructed**). Note that in release (optimized) compiler build, **RVO**¹ allows t to be **constructed directly in c** and no copy or move construction is needed
- $A \ d = \text{std::move}(a);$ $\Rightarrow$ **std::move** (in **<utility>**) can force an **rvalue** type. It produces an rvalue from t to indicate that the object t may be **moved from**. Hence d is **move constructed** (if available, else **copy constructed**)
- $b = a;$ $\Rightarrow$ a is an **lvalue** and hence, b is **copy assigned**
- $c = a + b;$ $\Rightarrow$ As above, c is **move assigned** (if available, else **copy assigned**) after **move construction** (if available, else **copy construction**). **Copy (move) construction** is eliminated by **RVO** in release build

¹Return Value Optimization (**RVO**) eliminates the temp. obj. created to hold a function's return value



Copy vs. Move : Vector

Module M49

Partha Pratim
Das

Objectives &
Outlines

Copying vs.
Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow
Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue
Vector

Implementing
Move Semantics

Module Summary

```
// C++ program with the copy and the move constructors
```

```
class C { int* data; // Declare the raw pointer as the data member of class
public:
    C(int d) {          // Constructor
        data = new int(d); // Declare object in the heap
        cout << "Ctor: " << d << endl;
    };
    C(const C& src) : myClass{ *src.data } { // Copy Constructor by delegation
        // Copying the data by making deep copy
        cout << "C-Ctor: " << *src.data << endl;
    }
    C(C&& src) : data{ src.data } noexcept { // Move Constructor
        cout << "M-Ctor: " << *src.data << endl;
        src.data = nullptr;
    }
    ~C() { // Destructor
        if (data != nullptr) // If pointer is not pointing to nullptr
            cout << "Dtor: " << *data << endl;
        else // If pointer is pointing to nullptr
            cout << "Dtor: " << "nullptr " << endl;
        delete data; // Free up the memory assigned to the data member of the object
    }
};
```



Copy vs. Move : Vector

Module M49

Partha Pratim Das

```
int main() { vector<C> v; // Create vector of C Class
            v.push_back(C{10}); // Inserting object of C class
            v.push_back(C{20});
        }
```

	Only Copy			Copy & Move		
	Debug	Release	Remark	Debug	Release	Remark
{ vector<C> v;						
// v.size() = 0	Ctor: 10	Ctor: 10		Ctor: 10	Ctor: 10	
v.push_back(C{10});	Ctor: 10	Ctor: 10	// Delegate			
// v.size() = 1	C-Ctor: 10	C-Ctor: 10	// C-Ctor	M-Ctor: 10	M-Ctor: 10	// Add 10 to v
	Dtor: 10	Dtor: 10		Dtor: nullptr	Dtor: nullptr	
// Move C{10}	Ctor: 20	Ctor: 20		Ctor: 20	Ctor: 20	
// for C{20}	Ctor: 10	Ctor: 10	// Delegate			
v.push_back(C{20});	C-Ctor: 10	C-Ctor: 10	// C-Ctor	M-Ctor: 10	M-Ctor: 10	// Move 10 in v
// v.size() = 2	Dtor: 10	Dtor: 10		Dtor: nullptr	Dtor: nullptr	
	Ctor: 20	Ctor: 20	// Delegate			
	C-Ctor: 20	C-Ctor: 20	// C-Ctor	M-Ctor: 20	M-Ctor: 20	// Add 20 to v
	Dtor: 20	Dtor: 20		Dtor: nullptr	Dtor: nullptr	
// End of scope	Dtor: 10	Dtor: 10	// Release	Dtor: 10	Dtor: 10	// Release
} // Release v	Dtor: 20	Dtor: 20	// Vector v	Dtor: 20	Dtor: 20	// Vector v



Copy vs. Move : Vector: Explanation

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue

Vector

Implementing Move Semantics

Module Summary

```
class C { int* data; /* raw pointer */ public:
C(int d); /*Ctor*/ C(const C& src); /*C-Ctor*/ C(C&& src); /*M-Ctor*/ ~C(); /*Dtor*/ };

• { vector<C> v; ⇒ v is default constructed as an empty vector of C. v.size() = 0
• v.push_back(C{10}); ⇒ Construct C{10}, copy/move & place in v[0], and destruct. v.size() = 1

Ctor: 10    /* Ctor for C{10} => t10, Temp.obj. and rvalue */ Ctor: 10
Ctor: 10    /* delegated from C-Ctor */
C-Ctor: 10  /* C-Ctor for t10 => v10 = v[0], lvalue to place in v */ M-Ctor: 10
Dtor: 10    /* Dtor for t10 */ Dtor: nullptr

• v.push_back(C{20}); ⇒ Construct C{20}. Copy/move v[0] and destruct old v[0]. Copy/move & place
  C{20} in v[1], and destruct. v.size() = 2

Ctor: 20    /* Ctor for C{20} => t20, Temp.obj. & rvalue */ Ctor: 20
Ctor: 10    /* delegated from C-Ctor */
C-Ctor: 10  /* C-Ctor for v10 => v10_1 = v[0], lvalue to place in v */ M-Ctor: 10
Dtor: 10    /* Dtor for v10 */ Dtor: nullptr
Ctor: 20    /* delegated from C-Ctor */
C-Ctor: 20  /* C-Ctor for t20 => v20 = v[1], lvalue to place in v */ M-Ctor: 20
Dtor: 20    /* Dtor for t20 */ Dtor: nullptr

• } ⇒ Automatic v going out of scope. Destruct v[0] and v[1]

Dtor: 10    /* Dtor for v10_1 = v[0] */ Dtor: 10
Dtor: 20    /* Dtor for v20 = v[1] */ Dtor: 20
```



Copy vs. Move : Vector: Performance Trade-off

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector
Swap

Deep vs. Shallow
Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue

Vector

Implementing
Move Semantics

Module Summary

- Since, class `C` has no default constructor, `vector<C> v` is constructed as an empty vector with `v.size() = 0`. Hence, every time a `push_back` (`insert` at the `end()`) is done, we need to expand the allocation of the vector by copying / moving the existing elements
- For `v.push_back(C{10})`, `C{10}` is constructed as a temporary object (`rvalue`). So, it needs to be copied / moved for `push_back` to the vector as `lvalue`. Same for `v.push_back(C{20})`
- Further, for `v.push_back(C{20})`, fresh allocation and copy / movement of existing element is needed for `push_back`
- To `push_back` the n^{th} element, we need to copy / move existing $n - 1$ elements. This means:
 - **Using Copy**
 - ▷ $n - 1$ resource allocations (`new int`) and de-allocations (`delete`)
 - ▷ For n elements this adds to $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2} = O(n^2)$ total allocations / de-allocations
 - **Using Move**
 - ▷ 0 resource allocations (`new int`) and de-allocations (`delete`)
 - ▷ For n elements this adds to $\sum_{i=0}^{n-1} 0 = 0$ total allocations / de-allocations. **Huge Benefit!**



Implementing Move Semantics

Module M49

Partha Pratim
Das

Objectives &
Outlines

Copying vs.
Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow
Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue
Vector

Implementing
Move Semantics

Module Summary

Implementing Move Semantics

Sources:

- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses
- [Scott Meyers on C++](#)



Implementing Move Semantics

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow

Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue

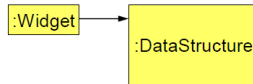
Vector

Implementing Move Semantics

Module Summary

- Move operations take source's value, but leave source in valid state:

```
class Widget {  
public:  
    Widget(Widget&& rhs) noexcept : pds(rhs.pds) // take source's value  
    { rhs.pds = nullptr; } // leave source in valid state  
  
    Widget& operator=(Widget&& rhs) noexcept {  
        delete pds; // get rid of current value  
        pds = rhs.pds; // take source's value  
        rhs.pds = nullptr; // leave source in valid state  
        return *this;  
    }  
    ...  
  
private:  
    struct DataStructure;  
    DataStructure *pds;  
};
```



- Easy for built-in types (for example, pointers). Trickier for UDTs...



Implementing Move Semantics

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue

Vector

Implementing Move Semantics

Module Summary

- `Widget`'s move `operator=` fails given move-to-self:

```
Widget w;  
w = std::move(w); // undefined behavior!
```

- It may be harder to recognize, of course:

```
Widget *pw1, *pw2;  
...  
*pw1 = std::move(*pw2); // undefined if pw1 == pw2
```

- C++11 condones this

- In contrast to copy `operator=`

- A fix is simple, if you are inclined to implement it:

```
Widget& Widget::operator=(Widget&& rhs) noexcept {  
    if (this == &rhs) return *this; // or assert(this != &rhs);  
    ...  
}
```




Implementing Move Semantics

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow

Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue

Vector

Implementing Move Semantics

Module Summary

- Part of C++11's `string` type:

```
string::string(const string&);    // copy constructor
string::string(string&&) noexcept; // move constructor
```

- An incorrect move constructor:

```
class Widget { std::string s;
public:
    Widget(Widget&& rhs) noexcept // move constructor
        : s(rhs.s) // compiles, but copies!
    { ... }
    ...
};
```

- `rhs.s` an **lvalue**, because it has a name
 - **Lvalueness** / **Rvalueness** orthogonal to type!
 - ▷ `ints` can be **lvalues** or **rvalues**, and **rvalue** references can, too.
 - `s` initialized by `string`'s *copy* constructor



Implementing Move Semantics

Module M49

Partha Pratim Das

Objectives & Outlines

Copying vs. Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue

Vector

Implementing Move Semantics

Module Summary

- Another example:

```
class WidgetBase { public:
    WidgetBase(const WidgetBase&);           // copy ctor
    WidgetBase(WidgetBase&&) noexcept;       // move ctor
    ...
};

class Widget: public WidgetBase { public:
    Widget(Widget&& rhs) noexcept           // move ctor
    : WidgetBase(rhs)                      // copies!
    { ... }
    ...
};
```

- `rhs` is an **lvalue**, because it has a name
 - Its declaration as `Widget&&` not relevant!



Module Summary

Module M49

Partha Pratim
Das

Objectives &
Outlines

Copying vs.
Moving

Return Value

Append Full Vector

Swap

Deep vs. Shallow
Copy

Performance Test

Rvalue & Move

Rvalue References

Copy vs. Move

Lvalue vs. Rvalue

Vector

Implementing
Move Semantics

Module Summary

- Understood the difference between Copying and Moving
- Understood the difference between Lvalue and Rvalue
- Learnt the advantages of Move in C++ using
 - Rvalue Reference
 - Move Semantics
 - Copy / Move Constructor / Assignment
 - Implementation of Move Semantics



Module M50

Partha Pratim
Das

Objectives &
Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges
Solution

`std::move`
Use
Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

Programming in Modern C++

Module M50: C++11 and beyond: General Features: Part 5: Rvalue and Move/2

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M50

Partha Pratim
Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment
Challenges
Solution

`std::move`

Use
Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

- Understood the difference between Copying and Moving
- Understood the difference between Lvalue and Rvalue
- Learnt the advantages of Move in C++ using
 - Rvalue Reference
 - Move Semantics
 - Copy / Move Constructor / Assignment
 - Implementation of Move Semantics



Module Objectives

Module M50

Partha Pratim
Das

Objectives &
Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges
Solution

`std::move`

Use
Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

- To learn to implement move semantics in user-defined classes
 - Challenges
 - Workaround using `std::move`
- To learn the use and implementation of `std::move`
- To put all the pieces together in a project to code move-enabled UDTs



Module Outline

Module M50

Partha Pratim
Das

Objectives &
Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges
Solution

`std::move`
Use

Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

- 1 Recap of Copy vs. Move and related Concepts
- 2 Move Semantics: How to code?
 - Simple Move Constructor and Assignment
 - Challenges
 - Solution
- 3 `std::move`
 - Use
 - Implementation
- 4 Move Semantics Project
 - ResMgr Class
 - MyResource Class
 - MyClass Class
- 5 Module Summary



Recap of Copy vs. Move and related Concepts

Module M50

Partha Pratim
Das

Objectives &
Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges
Solution

`std::move`

Use
Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

Recap of Copy vs. Move and related Concepts

Sources:

- **Module 49:** *C++11 and beyond: General Features: Part 4: Rvalue and Move/1*
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses
- [Scott Meyers on C++](#)



Copying vs. Moving: Recap (Module 49)

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and Assignment

Challenges
Solution

`std::move`

Use

Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

- C++ has always supported copying object state:
 - *Copy* constructors, *Copy* assignment operators
- C++11 adds support for requests to *Move* object state for performance improvement. Examples show the benefits of Move in several contexts including:
 - Return by value from functions
 - Appending to a full vector
 - Swapping two variables
 - Choice of Swallow Copy in place of Deep Copy when possible
 - ...
- C++11 adds the following for related optimization
 - Rvalue Reference
 - Move Semantics through *Move* constructors, *Move* assignment operators



Lvalues, Rvalues and Rvalue References: Recap (Module 49)

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges
Solution

`std::move`

Use

Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

- **Lvalues** *are generally things we can take the address of:*
 - In C, Expressions on *left-hand-side (LHS)* of an assignment
 - Named objects - variables
 - Legal to apply address of (&) operator
 - **Lvalue** references (T&)
- **Rvalues** *are generally things we cannot take the address of:*
 - In C, Expressions on *right-hand-side (RHS)* of an assignment
 - Unnamed (temporary) objects - expressions, return by value from functions, etc.
 - **Rvalue** references (T&&) *identify objects that may be moved from*
- Important for overloading resolution using **Lvalues** / **Rvalues**
- **Lvalues** may bind to *lvalue references*
- **Rvalues** may bind to *lvalue references to const*
- **Rvalues** may bind to *rvalue references to non-const*
- **Lvalues** may *not* bind to *rvalue references*



Move Semantics: How to code?

Module M50

Partha Pratim
Das

Objectives &
Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges
Solution

`std::move`

Use
Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

Move Semantics: How to code?

Sources:

- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses
- [Scott Meyers on C++](#)



Move Semantics: How to code?

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges
Solution

std::move
Use

Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

- Move operations take source's value, may change the source, but leave the source in valid state:

```
class MyResource { // Representative resource class with move support
    char *str = nullptr; // Resource pointer
public:
    // Move: Constructs by moving resource from rvalue reference source (rr)
    MyResource(MyResource&& rr) noexcept : str(rr.str) // Take the value of source rr
    { rr.str = nullptr; } // Changed but valid state set for the source

    // Move: Assigns by moving resource from rvalue reference source (rr)
    MyResource& operator=(MyResource&& rr) noexcept {
        delete [] str; // Release the current value
        str = rr.str; // Take the value of source rr
        rr.str = nullptr; // Changed but Valid state set for the source
        return *this; // Return the assigned object
    }
    ...
}
```

- Easy for built-in types like pointers above. Gets tricky for UDTs



Move Semantics: How to code?

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and Assignment

Challenges
Solution

`std::move`

Use

Implementation

Project

ResMgr Class

MyResource Class

MyClass Class

Module Summary

- Like Copy Assignment, `MyResource`'s move `operator=` may fail for move-to-self:

```
MyResource r;  
r = std::move(r); // undefined behavior! std::move changes r to an rvalue reference  
...             // std::move will be discussed later in this module
```

```
MyResource *p1, *p2;  
...  
*p1 = std::move(*p2); // undefined if p1 == p2
```

- Fix is simple like the guard used in copy assignment:

```
MyResource& MyResource::operator=(MyResource&& rr) noexcept {  
    if (this != &rr) {  
        ... // Do the move as above  
    }  
    // else assert(this != &rr); // un-comment to disallow move-to-self  
    return *this;  
}
```



Move Semantics: How to code?

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move Constructor and Assignment

Challenges

Solution

std::move

Use

Implementation

Project

ResMgr Class

MyResource Class

MyClass Class

Module Summary

- Next let us attempt to build a *move-enabled* `MyClass` that use an object of `MyResource` as a resource data member:

```
class MyClass { MyResource mRrc; // Resource object
public:
    MyClass(MyClass&& c) noexcept : // Move Constructor
        mRrc(c.mRrc) // Compiles, but actually copies using MyResource(MyResource&)
    { ... }
    MyClass& operator=(MyClass&& c) noexcept { // Move Assignment
        if (this != &c)
            { mRrc = c.mRrc; } // Compiles, but actually copies using
            return *this;      // MyResource::operator=(MyResource&)
        }
    ...
};
```

- `c.mRrc` is an *lvalue*, because it has a name
 - Lvalue-ness* / *Rvalue-ness* orthogonal to type!
 - `ints` can be *lvalues* or *rvalues*, and *rvalue* references can, too.
 - `mRrc` initialized by `MyResource`'s *copy* constructor / assignment



Move Semantics: How to code?

Module M50

Partha Pratim
Das

Objectives &
Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges

Solution

std::move

Use

Implementation

Project

ResMgr Class

MyResource Class

MyClass Class

Module Summary

- Next let us try to extend the design on an inheritance hierarchy where `MyClass` ISA `MyClassBase`

```
class MyClassBase { public:  
    MyClassBase(const MyClassBase&);           // Copy Constructor  
    MyClassBase(MyClassBase&&) noexcept;       // Move Constructor  
    ...  
};  
  
class MyClass: public MyClassBase { public:  
    MyClass(MyClass&& c) noexcept // Move Constructor  
        : MyClassBase(c) // Compiles, but actually copies using MyClassBase(MyClassBase&)  
        { ... }  
    ...  
};
```

- `c` is an **lvalue**, because it has a name
 - Its declaration as `MyClass&&` not relevant!
- Similar issue will happen for `MyClass::operator=(MyClass&&)`



Move Semantics: How to code?

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and Assignment

Challenges
Solution

`std::move`
Use

Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

- **How to solve the problems in the coding of move semantics for UDTs?**
- In general, in a UDT, there are two kinds of constituent objects:
 - Data Member like `c.mRrc` // `MyClass c;`
 - Base Class part like `(MyClassBase&)c` // `MyClass: public MyClassBase`
- While we need to copy or move, we use the constructor or assignment operators of the underlying classes:
 - Construction: `mRrc(c.mRrc)` and `MyClassBase(c)`
 - Assignment: `mRrc = c.mRrc` and `MyClassBase::operator=(c)`
- For **copy**, we use the **copy constructor / assignment operator**
- For **move**, we need to use the **move constructor / assignment operator** to optimize resource handling
 - This means for the above four instances of the respective operators, the sources (`c.mRrc` or `c`) must be **rvalues**. But they are available by name as **lvalues**
- Hence, **we need a mechanism to convert an lvalue to an rvalue**
- `std::move` in `<utility>` provides for this



std::move

Module M50

Partha Pratim
Das

Objectives &
Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges
Solution

std::move

Use

Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

std::move

Sources:

- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses
- [Scott Meyers on C++](#)
- [Quick Q: What's the difference between std::move and std::forward?](#), isocpp.org
- [On the Superfluosness of std::move](#) – Scott Meyers, isocpp.org, 2012
- [std::move](#), cppreference.com



Explicit Move Requests

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges
Solution

`std::move`

Use

Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

- To request a move on an **lvalue**, use `std::move` from `<utility>`:

```
class MyClassBase { ... };
class MyClass: public MyClassBase { public:
    MyClass(MyClass&& c) noexcept : // Move Constructor
        MyClassBase(std::move(c)), // Request Move for base class part
        mRrc(std::move(c.mRrc))    // Request Move for data member
    { ... }
    MyClass& operator=(MyClass&& c) noexcept { // Move Assignment
        if (this != &c) {
            MyClassBase::operator=(std::move(c)); // Request Move for base class part
            mRrc = std::move(c.mRrc);             // Request Move for data member
        }
        return *this;
    }
    ...
};
```

- `std::move` turns **lvalues** into **rvalues**
 - The overloading rules do the rest



Explicit Move Requests

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and Assignment

Challenges
Solution

`std::move`

Use

Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

- `std::move` uses *implicit type deduction*

Consider:

```
template<typename It>
void someAlgorithm(It begin, It end) {
    // permit move from *begin to temp

    // static_cast version
    auto temp1 = static_cast<typename std::iterator_traits<It>::value_type&&>(*begin);

    // C-style cast version
    auto temp2 = (typename std::iterator_traits<It>::value_type&&)*begin;

    // std::move version
    auto temp3 = std::move(*begin);
    ...
}
```

- Great convenience by using `std::move`



Reference Collapsing in Templates

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and Assignment

Challenges
Solution

std::move

Use

Implementation

Project

ResMgr Class

MyResource Class

MyClass Class

Module Summary

- In C++03, given

```
template<typename T> void f(T& param);
int x;
f<int&>(x);
```

// T is int&
- `f` is initially instantiated as

```
void f(int& & param);
```

// reference to reference
- C++03's reference-collapsing rule says
 - `T& & => T&`
- So, after reference collapsing, `f`'s instantiation is actually: `void f(int& param);`
- C++11's rules take rvalue references into account:
 - `T& & => T&` *// from C++03*
 - `T&& & => T&` *// new for C++11*
 - `T& && => T&` *// new for C++11*
 - `T&& && => T&&` *// new for C++11*
- Summary:
 - *Reference collapsing involving a `&` is always `T&`*
 - *Reference collapsing involving only `&&` is `T&&`*



std::move: Return Type

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and Assignment

Challenges
Solution

std::move

Use

Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

- To guarantee an **rvalue** return type, `std::move` does this:

```
template<typename T>
typename std::remove_reference<T>::type&&
move(MagicReferenceType obj) noexcept {
    return obj;
}
```

- Recall that a `T&` return type would be an **lvalue**!
- Hence:

```
int x;
std::move<int&>(x); // calls remove_reference<int&>::type&& std::move(/*...*/)
                  // => int&& std::move(/*...*/)

```

- Without `std::remove_reference`, `move<int&>` would return `int&`



std::move: Parameter Type

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and Assignment

Challenges
Solution

std::move

Use

Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

- Must be a non-**const** reference, because we want to move its value
- An **lvalue** reference does not work, because **rvalues** cannot bind to them:

```
TVec createTVec();           // as before
TVec&& std::move(TVec& obj) noexcept; // possible move instantiation
std::move(createTVec());     // error!
```

- An **rvalue** reference does not, either, as **lvalues** cannot bind to them:

```
TVec&& std::move(TVec&& obj) noexcept; // possible move instantiation
TVec vt;
std::move(vt);                        // error!
```

- What **std::move** needs:
 - For **lvalue** arguments, a parameter type of **T&**
 - For **rvalue** arguments, a parameter type of **T&&**



std::move: Parameter Type: Solution by Overloading?

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and Assignment

Challenges
Solution

std::move

Use

Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

- Overloading could solve the problem:

```
template<typename T>
typename std::remove_reference<T>::type&&
move(T& lvalue) noexcept {
    return static_cast<std::remove_reference<T>::type&&>(lvalue);
}
template<typename T>
typename std::remove_reference<T>::type&&
move(T&& rvalue) noexcept {
    return static_cast<std::remove_reference<T>::type&&>(rvalue);
}
```

- But the *perfect forwarding problem*¹ would remain:
 - To forward *n* arguments to another function we would need 2^n overloads!
- *Rvalue references* aimed at both `std::move` and *perfect forwarding*

¹ *Perfect Forwarding Problem will be discussed in a later module*



std::move: T&& Parameter Deduction in Templates

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and Assignment

Challenges
Solution

std::move

Use

Implementation

Project

ResMgr Class

MyResource Class

MyClass Class

Module Summary

- Given

```
template<typename T> void f(T&& param); // note non-const rvalue reference
```

- T's deduced type depends on what is passed to param:

- **Lvalue** $\Rightarrow$ T is an lvalue reference (T&)
- **Rvalue** $\Rightarrow$ T is a non-reference (T)

- In conjunction with reference collapsing:

```
int x;
f(x);           // lvalue: generates f<int&>(int& &&), calls f<int&>(int&)
f(10);          // rvalue: generates/calls f<int>(int&&)

TVec vt;        // typedef vector<int> TVec;
                // TVec createTVec();

f(vt);          // lvalue: generates f<TVec&>(TVec& &&), calls f<TVec&>(TVec&)
f(createTVec()); // rvalue: generates/calls f<TVec>(TVec&&)
```




std::move: Implementation

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and Assignment

Challenges
Solution

std::move

Use

Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

- `std::move`'s parameter is thus `T&&`:

```
template<typename T>
typename std::remove_reference<T>::type&&
    move(T&& obj) noexcept {
        return obj;
    }
```

- This is almost correct. Problem:
 - `obj` is an **lvalue** (It has a name)
 - `move`'s return type is an **rvalue** reference
 - **Lvalues** cannot bind to **rvalue** references

- A cast eliminates the problem to give a correct implementation

```
template<typename T>
typename std::remove_reference<T>::type&&
    move(T&& obj) noexcept {
        using ReturnType = typename std::remove_reference<T>::type&&;
        return static_cast<ReturnType>(obj);
    }
```



T&& Parameters in Templates

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and Assignment

Challenges
Solution

`std::move`

Use

Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

- Compare conceptual and actual `std::move` declarations:

```
template<typename T>                                // conceptual
T&&
move(MagicReferenceType obj) noexcept;
```

```
template<typename T>                                // actual
typename std::remove_reference<T>::type&&
move(T&& obj) noexcept;
```

- `T&&` really is a magic² reference type!
 - For **lvalue** arguments, `T&&` becomes `T&` => **lvalues** can bind
 - For **rvalue** arguments, `T&&` remains `T&&` => **rvalues** can bind
 - For **const/volatile** arguments, **const/volatile** becomes part of `T`
 - `T&&` parameters can bind *anything*

² *Universal Reference will be discussed in a later module*



Move Semantics Project

Module M50

Partha Pratim
Das

Objectives &
Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges
Solution

`std::move`

Use

Implementation

Project

`ResMgr` Class

`MyResource` Class

`MyClass` Class

Module Summary

Move Semantics Project



Move Semantics Project

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges
Solution

std::move

Use

Implementation

Project

ResMgr Class

MyResource Class

MyClass Class

Module Summary

- To put the pieces of the Move Semantics puzzle together, we present a complete project with classes having resources of POD³ and UDT
- We also code a resource management helper class ([ResMgr](#)) to *track the objects (resources) created and released dynamically*. This will help us to *quantify the benefits of move over copy*
- All functions are tracked with messages to understand *object lifetimes under copy and move*
- Using [ResMgr](#), we present applications to compare the performance of the *Copy & Move* version of the classes against the *Copy Only* version
- The classes are (skeletons of [MyResource](#) and [MyClass](#) used earlier in the module):
 - [ResMgr](#): *Resource Management Helper Class*. It has static member functions to [Create\(\)](#), [Release\(\)](#) resources and print statistics ([Stat\(\)](#))
 - [MyResource](#): *Resource Class with POD resource (char*)*. It has usual class members, overloaded output operator and a global function to call and return by value
 - [MyClass](#): *Resource Class with UDT resource (MyResource)*. It has usual class members, overloaded output operator and a global function to call and return by value
- **The codes may be used to code move semantics in any project you develop**

³Plain Old Data (POD) refers to built-in types



Move Semantics: ResMgr: Resource Management Helper Class

Module M50

Partha Pratim
Das

Objectives &
Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges
Solution

std::move

Use

Implementation

Project

ResMgr Class

MyResource Class

MyClass Class

Module Summary

```
#include <iostream>
#include <cstring>
using namespace std;

class ResMgr { // Resource Management class to track creation and release of resources
static unsigned int nCreated, nReleased; // Counters for created & released resources
public:
    ResMgr() { } // Constructor to be called before main
    ~ResMgr() { // Destructor to be called after main
        cout << "\n\nResources Created = " << nCreated << endl;
        cout << "Resources Released = " << nReleased << endl;
    }
    inline static char *Create(const char *s) // Create a resource from s
    { return (s) ? ++nCreated, strdup(s) : nullptr; } // If s is not null, copy & increment counter
    inline static void Release(char *s) // Release the resource held by s
    { (s) ? free(s), ++nReleased : 0; } // If s is not null, increment counter & free resource
    inline static void Stat() // Print stats for resources created and released
    { cout << " Stat = (" << nCreated << ", " << nReleased << ")\n\n"; }
};

unsigned int ResMgr::nCreated = 0; // Define and initialize Counters
unsigned int ResMgr::nReleased = 0;

ResMgr m; // Static resource manager instance
// Created before call to main(). Destroyed after return from main()
```



Move Semantics: MyResource: POD Resource Class - *Copy Only*

Module M50

Partha Pratim
Das

Objectives &
Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges
Solution

std::move

Use

Implementation

Project

ResMgr Class

MyResource Class

MyClass Class

Module Summary

```
class MyResource { // Representative resource class with copy-only support
    char *str = nullptr; // Resource pointer
public:
    MyResource(const char* s = nullptr) : str(ResMgr::Create(s)) // Param. & Defa. Ctor
    { cout << "Ctor[R] "; } // Creates resource
    MyResource(const MyResource& s) : str(ResMgr::Create(s.str)) // Copy Ctor
    { cout << "C-Ctor[R] "; } // Copy-Creates resource
    MyResource& operator=(const MyResource& s) { cout << "C=[R] "; // Copy Assignment
        if (this != &s) { ResMgr::Release(str); str = ResMgr::Create(s.str); }
        return *this; // Releases and Copy-Creates resource
    }
    ~MyResource() // Destructor
    { cout << "Dtor[R] "; ResMgr::Release(str); } // Releases resource
    friend ostream& operator<<(ostream& os, const MyResource& s) { // Streams resource value
        cout << ((s.str) ? s.str : "null"); return os; // Streams "null" for nullptr (no resource)
    }
};

MyResource f(MyResource s) // Global function
{ cout << "f[R] "; return s; } // Uses call-by-value & return-by-value
```



Move Semantics: Application using *Copy Only* MyResource

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges

Solution

std::move

Use

Implementation

Project

ResMgr Class

MyResource Class

MyClass Class

Module Summary

```
// ResMgr m is constructed here
int main() { // Appl. to check resource behavior for copy-only support through copy ctor & assignment
    MyResource r1{ "ppd" }; // Ctor[R] r1=ppd Stat = (1, 0)
    cout << "r1=" << r1; ResMgr::Stat(); // r1 constructed with parameter

    MyResource r2{ r1 }; // C-Ctor[R] r2=ppd r1=ppd Stat = (2, 0)
    cout << "r2=" << r2 << " r1=" << r1; ResMgr::Stat(); // r2 copy constructed from r1

    MyResource r3{ f(r2) }; // C-Ctor[R] f[R] C-Ctor[R] Dtor[R] r3=ppd r2=ppd Stat = (4, 1)
    cout << "r3=" << r3 << " r2=" << r2; ResMgr::Stat(); // r3 C-Ctor from f(r2): C-Ctor / Dtor for param

    r1 = r2; // C=[R] r1=ppd r2=ppd Stat = (5, 2)
    cout << "r1=" << r1 << " r2=" << r2; ResMgr::Stat(); // r1 copy assigned from r2

    MyResource r4; // Ctor[R] r4=null Stat = (5, 2)
    cout << "r4=" << r4; ResMgr::Stat(); // r4 default constructed

    r4 = f(r3); // C-Ctor[R] f[R] C-Ctor[R] C=[R] Dtor[R] Dtor[R] r4=ppd r3=ppd Stat = (8, 4)
    cout << "r4=" << r4 << " r3=" << r3; ResMgr::Stat(); // r4 C= from f(r3): trace debug to understand
} // m.~ResMgr is called after the destruction of local automatic objects to print the final statistics
// Dtor[R] Dtor[R] Dtor[R] Dtor[R]
// Resources Created = 8 Resources Released = 8 // printed from m.~ResMgr
```

- Note that ResMgr m is a global static object that is *created before and destroyed after* main()
- Track function call messages to understand the lifetimes of object



Move Semantics: MyResource: POD Resource Class

Copy & Move

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and Assignment

Challenges
Solution

std::move
Use

Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

```
class MyResource { // Representative resource class with copy-and-move support
    char *str = nullptr; // Resource pointer
public:
    MyResource(const char* s = nullptr) : str(ResMgr::Create(s)) // Param. & Defa. Ctor
    { cout << "Ctor[R] "; } // Creates resource
    MyResource(const MyResource& s) : str(ResMgr::Create(s.str)) // Copy Ctor
    { cout << "C-Ctor[R] "; } // Copy-Creates resource
    MyResource(MyResource&& s) noexcept : str(s.str) // Move Ctor
    { cout << "M-Ctor[R] "; s.str = nullptr; } // Moves resource
    MyResource& operator=(const MyResource& s) { cout << "C=[R] "; // Copy Assignment
        if (this != &s) { ResMgr::Release(str); str = ResMgr::Create(s.str); }
        return *this; // Releases and Copy-Creates resource
    }
    MyResource& operator=(MyResource&& s) noexcept { cout << "M=[R] "; // Move Assignment
        if (this != &s) { ResMgr::Release(str); str = s.str; s.str = nullptr; }
        return *this; // Releases and Moves resource
    }
    ~MyResource() // Destructor
    { cout << "Dtor[R] "; ResMgr::Release(str); } // Releases resource
    friend ostream& operator<<(ostream& os, const MyResource& s) { // Streams resource value
        cout << ((s.str) ? s.str : "null"); return os; // Streams "null" for nullptr (no resource)
    }
};

MyResource f(MyResource s) // Global function
{ cout << "f[R] "; return s; } // Uses call-by-value & return-by-value

Programming in Modern C++
```




Move Semantics: Application using *Copy & Move* MyResource

Module M50

Partha Pratim
Das

Objectives &
Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges
Solution

std::move

Use
Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

```
int main() { // Application to check resource behavior for copy-and-move support through
              // copy construction / assignment and move construction / assignment
    MyResource r1{ "ppd" }; // Ctor[R] r1=ppd Stat = (1, 0)
    cout << "r1=" << r1; ResMgr::Stat();

    MyResource r2{ r1 }; // C-Ctor[R] r2=ppd r1=ppd Stat = (2, 0)
    cout << "r2=" << r2 << " r1=" << r1; ResMgr::Stat();

    MyResource r3{ f(r2) }; // C-Ctor[R] f[R] M-Ctor[R] Dtor[R] r3=ppd r2=ppd Stat = (3, 0)
    cout << "r3=" << r3 << " r2=" << r2; ResMgr::Stat(); // r3 M-Ctor from f(r2): C-Ctor / Dtor for param

    r1 = r2; // C=[R] r1=ppd r2=ppd Stat = (4, 1)
    cout << "r1=" << r1 << " r2=" << r2; ResMgr::Stat();

    MyResource r4; // Ctor[R] r4=null Stat = (4, 1)
    cout << "r4=" << r4; ResMgr::Stat();

    r4 = f(r3); // C-Ctor[R] f[R] M-Ctor[R] M=[R] Dtor[R] Dtor[R] r4=ppd r3=ppd Stat = (5, 1)
    cout << "r4=" << r4 << " r3=" << r3; ResMgr::Stat(); // r4 M= from f(r3): Note M-Ctor in f(r3)
}
// Dtor[R] Dtor[R] Dtor[R] Dtor[R]
// Resources Created = 5 Resources Released = 5
```

- Compared to copy-only, we created and released $(8 - 5) = 3$ resources less with copy-and-move



Move Semantics: MyClass: UDT Resource Class - *Copy & Move* (broken!)

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges
Solution

std::move

Use

Implementation

Project

ResMgr Class

MyResource Class

MyClass Class

Module Summary

```
class MyClass { MyResource mRrc; // Resource object
public:
    MyClass() : mRrc("") // Defa. Ctor
    { cout << "D-Ctor[C] "; }
    MyClass(const MyResource& r) : mRrc(r) // Param. Ctor
    { cout << "Ctor[C] "; }
    MyClass(const MyClass& c) : mRrc(c.mRrc) // Copy Ctor
    { cout << "C-Ctor[C] "; }
    MyClass(MyClass&& c) noexcept : mRrc(c.mRrc) // Move Ctor
    { cout << "M-Ctor[C] "; }
    MyClass& operator=(const MyClass& c) { cout << "C=[C] "; // Copy Assignment
        if (this != &c) { mRrc = c.mRrc; }
        return *this;
    }
    MyClass& operator=(MyClass&& c) noexcept { cout << "M=[C] "; // Move Assignment
        if (this != &c) { mRrc = c.mRrc; }
        return *this;
    }
    ~MyClass() /* Destructor */ { cout << "Dtor[C] "; }
    friend ostream& operator<<(ostream& os, const MyClass& c) // Streams resource value
    { cout << c.mRrc; return os; }
};

MyClass f(MyClass s) // Global function
{ cout << "f[C] "; return s; } // Uses call-by-value & return-by-value
```



Move Semantics: Application using *Copy & Move* MyClass

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges
Solution

std::move

Use

Implementation

Project

ResMgr Class

MyResource Class

MyClass Class

Module Summary

```
int main() {
    MyResource r1{ "ppd" }; // Ctor[R] r1=ppd Stat = (1, 0)
    cout << "r1=" << r1; ResMgr::Stat();

    MyClass c1{ r1 }; // C-Ctor[R] Ctor[C] c1=ppd Stat = (2, 0)
    cout << "c1=" << c1; ResMgr::Stat();

    MyClass c2{ f(c1) }; // C-Ctor[R] C-Ctor[C] f[C] C-Ctor[R] M-Ctor[C] Dtor[C] Dtor[R]
                        // c2=ppd c1=ppd Stat = (4, 1)
                        // c2 C-Ctor[C] from f(c1). Calls M-Ctor[C] yet copies resource
    cout << "c2=" << c2 << " c1=" << c1; ResMgr::Stat();

    c1 = f(c2); // C-Ctor[R] C-Ctor[C] f[C] C-Ctor[R] M-Ctor[C] M=[C] C=[R]
              // Dtor[C] Dtor[R] Dtor[C] Dtor[R]
              // c1=ppd c2=ppd Stat = (7, 4)
              // c1 C=[C] from f(c2). Calls M=[C] yet copies resource
    cout << "c1=" << c1 << " c2=" << c2; ResMgr::Stat();
}
// Dtor[C] Dtor[R] Dtor[C] Dtor[R] Dtor[R]
// Resources Created = 7 Resources Released = 7
```



Move Semantics: MyClass: : UDT Resource Class - Copy & Move (fixed!)

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and Assignment

Challenges
Solution

std::move

Use

Implementation

Project

ResMgr Class

MyResource Class

MyClass Class

Module Summary

```
class MyClass { MyResource mRrc; // Resource object
public:
    MyClass() : mRrc("") // Defa. Ctor
    { cout << "D-Ctor[C] "; }
    MyClass(const MyResource& r) : mRrc(r) // Param. Ctor
    { cout << "Ctor[C] "; }
    MyClass(const MyClass& c) : mRrc(c.mRrc) // Copy Ctor
    { cout << "C-Ctor[C] "; }
    MyClass(MyClass&& c) noexcept : mRrc(std::move(c.mRrc)) // Move Ctor
    { cout << "M-Ctor[C] "; }
    MyClass& operator=(const MyClass& c) { cout << "C=[C] "; // Copy Assignment
        if (this != &c) { mRrc = c.mRrc; }
        return *this;
    }
    MyClass& operator=(MyClass&& c) noexcept { cout << "M=[C] "; // Move Assignment
        if (this != &c) { mRrc = std::move(c.mRrc); }
        return *this;
    }
    ~MyClass() /* Destructor */ { cout << "Dtor[C] "; }
    friend ostream& operator<<(ostream& os, const MyClass& c) // Streams resource value
    { cout << c.mRrc; return os; }
};

MyClass f(MyClass s) // Global function
{ cout << "f[C] "; return s; } // Uses call-by-value & return-by-value
```



Move Semantics: Application using *Copy & Move* MyClass

Module M50

Partha Pratim Das

Objectives & Outlines

Recap

Move Semantics

Simple Move
Constructor and Assignment

Challenges
Solution

std::move

Use
Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

```
int main() {
    MyResource r1{ "ppd" }; // Ctor[R] r1=ppd Stat = (1, 0)
    cout << "r1=" << r1; ResMgr::Stat();

    MyClass c1{ r1 }; // C-Ctor[R] Ctor[C] c1=ppd Stat = (2, 0)
    cout << "c1=" << c1; ResMgr::Stat();

    MyClass c2{ f(c1) }; // C-Ctor[R] C-Ctor[C] f[C] M-Ctor[R] M-Ctor[C] Dtor[C] Dtor[R]
                        // c2=ppd c1=ppd Stat = (3, 0)
                        // c2 C-Ctor[C] from f(c1). Calls M-Ctor[C] and does not copy. FIXED
    cout << "c2=" << c2 << " c1=" << c1; ResMgr::Stat();

    c1 = f(c2); // C-Ctor[R] C-Ctor[C] f[C] M-Ctor[R] M-Ctor[C] M=[C] M=[R]
              // Dtor[C] Dtor[R] Dtor[C] Dtor[R]
              // c1=ppd c2=ppd Stat = (4, 1)
              // c1 C=[C] from f(c2). Calls M-Ctor[C] and does not copy. FIXED
    cout << "c1=" << c1 << " c2=" << c2; ResMgr::Stat();
}
// Dtor[C] Dtor[R] Dtor[C] Dtor[R] Dtor[R]
// Resources Created = 4 Resources Released = 4
```

- Compared to copy-and-move without std::move, we created and released $(7 - 4) = 3$ resources less with std::move



Module Summary

Module M50

Partha Pratim
Das

Objectives &
Outlines

Recap

Move Semantics

Simple Move
Constructor and
Assignment

Challenges
Solution

`std::move`

Use
Implementation

Project

ResMgr Class
MyResource Class
MyClass Class

Module Summary

- Learnt to implement move semantics in UDTs using `std::move`
- Understood the use and implementation of `std::move`
- Studied a project to code move-enabled UDTs



Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

Programming in Modern C++

Tutorial T10: How to optimize C++11 programs using Rvalue and Move Semantics?

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Tutorial Objectives

Tutorial T10

Partha Pratim
Das

Objective & Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- To understand optimization by copy elision
- To understand copy / move optimization by Rvalues and Move Semantics



Tutorial Outline

Tutorial T10

Partha Pratim
Das

Objective & Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- 1 Optimizing C++11 Programs
- 2 Copy Elision
 - Copy Initialization
 - Return Value Optimization (RVO)
 - Language Specification
- 3 Sorting Objects
 - Copy Support
 - Statistics Support
 - Move Support
 - Summary
 - Project Codes
 - Problems
- 4 Tutorial Summary



Optimizing C++11 Programs

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

Optimizing C++11 Programs

Sources:

- [Move semantics and rvalue references in C++11](#), cprogramming, Alex Allain, 2019
- [Copy elision](#), wikipedia



Optimizing C++11 Programs

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- C++ has always produced fast programs
- Unfortunately, until C++11, there has been an obstinate wart that slows down many C++ programs:
 - the *creation of temporary objects*
- Sometimes these temporary objects can be optimized away by the compiler by *copy elision*¹ (the return value optimization, for example). But this is not always the case, and it can result in expensive object copies
- Copy elision (or omission) depends primarily on identification of *rvalues* by the compiler and can be optimized away
- In addition to what the compiler can do, we can reduce copies by explicitly marking *rvalues* in the code by Rvalue references and by providing the move operations along with the copy operations (if needed)
- We first elucidate some common scenarios of copy elision that the language standard specifies and the compiler exploits for optimization
- Next we show through a small sorting project how the programmer can expose good move opportunities for the compiler to optimize copies

¹compiler optimization technique that eliminates unnecessary copying of objects



Copy Elision

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

Copy Elision

Sources:

- [Copy elision](#), cppreference
- [Copy elision in C++](#), geeksforgeeks, 2017
- [Copy elision](#), wikipedia



Copy Elision

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- In C++ programming, **copy elision**² refers to a *compiler optimization technique that eliminates unnecessary copying of objects*
- The C++ language standard generally allows implementations to perform any optimization, provided the resulting program's observable behavior is the same as if, that is pretending, the program were executed exactly as mandated by the standard.
- Beyond that, the standard also describes a few situations where copying can be eliminated even if this would alter the program's behavior
 - the most common being the *return value optimization*
 - Another widely implemented optimization, described in the C++ standard, is when a temporary object of class type is copied to an object of the same type. As a result
 - ▷ *copy-initialization* is usually equivalent to *direct-initialization* in terms of performance, but semantically,
 - ▷ copy-initialization still requires an accessible copy constructor
- The optimization cannot be applied to a temporary object that has been bound to a reference

²*elision* is the omission of a sound or syllable when speaking (as in I'm, let's)



Copy Elision: Copy Initialization

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

Copy Elision: Copy Initialization



Copy Elision: Copy Initialization

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- What will be the output?

```
#include <iostream>
int n = 0;
struct C {
    explicit C(int) { }
    C(const C&) { ++n; } // the copy constructor has a visible side effect
};                               // it modifies an object with static storage duration
int main() {
    C c1(42); // direct-initialization, calls C(int). c1 is lvalue
    C c2 = C(77); // copy-initialization. C(77) by C(int) is rvalue, skips C(const C&)
    std::cout << n << std::endl; // prints 0 if the copy was elided, 1 otherwise // 0
}
```

- Interestingly both GCC-C++ and MSVC++ and print 0 even in debug build
- Copy constructor `C::C(const C&)` is not even invoked
- If you think this is because `C::C(const C&)` does not do anything meaningful for the object, check the next version



Copy Elision: Copy Initialization

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- What will be the output?

```
#include <iostream>
int n = 0;
struct C { int i;
    explicit C(int i) : i(i) { std::cout << i << ' '; }
    C(const C& c) : i(c.i) { std::cout << ++i << ' '; ++n; }
    ~C() { std::cout << "~" << i << ' '; }
};
int main() {
    C c1(42);           // direct-init., calls C(int). c1 is lvalue           // 42
    C c2 = C(77);       // copy-init. C(77) by C(int) is rvalue, skips C(const C&) // 77
    std::cout << n << std::endl; // prints 0 if the copy was elided, 1 otherwise // 0
} // ~77 ~42
```

- `C::C(const C&)` is just not invoked!
- Yet, if you comment the copy constructor and explicitly delete it (`C(const C&) = delete;`) so that no free copy constructor is provided, C++11 will give **error: use of deleted function 'C::C(const C&)'**



Copy Elision: Copy Initialization

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- Let us construct an object from an **lvalue**

```
#include <iostream>
int n = 0;
struct C { int i;
    explicit C(int i) : i(i) { std::cout << i << ' '; }
    C(const C& c) : i(c.i) { std::cout << ++i << ' '; ++n; }
    ~C() { std::cout << "~" << i << ' '; }
};
int main() {
    C c1(42);           // direct-init., calls C(int). c1 is lvalue           // 42
    C c2 = C(77);       // copy-init. C(77) by C(int) is rvalue, skips C(const C&) // 77
    C c3 = c1;          // copy-init., calls C(const C&) as c1 is lvalue      // 43
    std::cout << n << std::endl; // prints 0 if the copy was elided, 1 otherwise // 1
} // ~43 ~77 ~42
```



Copy Elision: Copy Initialization

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- Using `-fno-elide-constructors` option to disable copy-elision:

```
#include <iostream>
int n = 0;
struct C { int i;
    explicit C(int i) : i(i) { std::cout << i << ' '; }
    C(const C& c) : i(c.i) { std::cout << ++i << ' '; ++n; }
    ~C() { std::cout << "~" << i << ' '; }
};
int main() {
    C c1(42);           // direct-init., calls C(int). c1 is lvalue           // 42
    C c2 = C(77);       // copy-init. C(77) by C(int) is rvalue, skips C(const C&) // 77 78 ~77
    C c3 = c1;          // copy-init., calls C(const C&) as c1 is lvalue       // 43
    std::cout << n << std::endl; // prints 0 if the copy was elided, 1 otherwise // 2
} // ~43 ~78 ~42
```



Copy Elision: Return Value Optimization (RVO)

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

Copy Elision: Return Value Optimization (RVO)



Copy Elision: Return Value Optimization (RVO)

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- Similar behaviour would be observed through function return by direct construction:

```
#include <iostream>
int n = 0;
struct C { int i;
    explicit C(int i) : i(i) { std::cout << i << ' '; }
    C(const C& c) : i(c.i) { std::cout << ++i << ' '; ++n; }
    ~C() { std::cout << "~" << i << ' '; }
};

C f(int i) {
    return C(i); // directly constructed object by C(int): C(i) is rvalue
} // rvalue C(i) is to be copy constructed by C(const C&) to be returned as rvalue. Skipped

C g(int i) {
    C c(i); // directly constructed object: c is lvalue needs C(int)
    return c; // return object constructed from c by C(const C&) to be returned as rvalue
}

int main() {
    f(19); // f(19) is rvalue - unused and destructed // 19 ~19
    g(35); // f(19) is rvalue - unused and destructed // 35 ~35
    std::cout << n << std::endl; // prints 0 if the copy was elided, 1 otherwise // 0
}
```



Copy Elision: Return Value Optimization (RVO)

- Similar behaviour is also observed if the return value is used in initialization without being discarded – however, the destruction order changes:

```
#include <iostream>
int n = 0;
struct C { int i;
    explicit C(int i) : i(i) { std::cout << i << ' '; }
    C(const C& c) : i(c.i) { std::cout << ++i << ' '; ++n; }
    ~C() { std::cout << "~" << i << ' '; }
};
C f(int i) {
    return C(i); // directly constructed object by C(int): C(i) is rvalue
} // rvalue C(i) is to be copy constructed by C(const C&) to be returned as rvalue. Skipped
C g(int i) {
    C c(i); // directly constructed object: c is lvalue needs C(int)
    return c; // return object constructed from c by C(const C&) to be returned as rvalue
}
int main() {
    C c1 = f(19); // copy-init. f(19) by C(int) is rvalue, skips C(const C&) // 19
    C c2 = g(35); // copy-init. g(35) by C(int) is rvalue, skips C(const C&) // 35
    std::cout << n << std::endl; // prints 0 if the copy was elided, 1 otherwise // 0
} // ~35 ~19
Programming in Modern C++
```

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary



Copy Elision: Return Value Optimization (RVO)

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- Using `-fno-elide-constructors` option to disable copy-elision:

```
#include <iostream>
int n = 0;
struct C { int i;
    explicit C(int i) : i(i) { std::cout << i << ' '; }
    C(const C& c) : i(c.i) { std::cout << ++i << ' '; ++n; }
    ~C() { std::cout << "~" << i << ' '; }
};

C f(int i) {
    return C(i); // directly constructed object by C(int): C(i) is rvalue
} // rvalue C(i) is to be copy constructed by C(const C&) to be returned as rvalue. Skipped

C g(int i) {
    C c(i); // directly constructed object: c is lvalue needs C(int)
    return c; // return object constructed from c by C(const C&) to be returned as rvalue
}

int main() {
    C c1 = f(19); // copy-init. f(19) by C(int) is rvalue, skips C(const C&) // 19 20 ~19 21
    C c2 = g(35); // copy-init. g(35) by C(int) is rvalue, skips C(const C&) // 35 36 ~35 37
    std::cout << n << std::endl; // prints 0 if the copy was elided, 1 otherwise // 4
} // ~37 ~21
```



Copy Elision: Language Specification

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

Copy Elision: Language Specification

Sources:

- [Copy elision](#), cppreference



Mandatory Elision: Copy / Move Operations

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- Under the following circumstances, the compilers are required to omit the copy and move construction of class objects, even if the copy / move constructor and the destructor have observable side-effects
- *Objects are constructed directly into the storage where they would be copied / moved to*
- *The copy / move constructors need not be present or accessible:*
 - In a return statement, when the operand is a **prvalue** of the same class type (ignoring cv-qualification) as the return type:

```
T f() {  
    return T();  
}
```

`f();` // only one call to default constructor of T

- More are specified for C++17



Non-Mandatory Elision: Copy / Move Operations

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- Under the following circumstances, the compilers are *permitted, but not required* to omit the copy and move construction of class objects, *even if the copy / move constructor and the destructor have observable side-effects*
- *Objects are constructed directly into the storage where they would be copied / moved to*
- **This is an optimization:** *even when it takes place and the copy / move constructor is not called, it still must be present and accessible (as if no optimization happened at all), otherwise the program is ill-formed:*
 - In a *return statement*, when the operand is a named object (and not a function or a catch clause param) with automatic storage duration, and which is of the same class type (ignoring cv-qualification) as the function return type. This variant of copy elision is known as **Named Return Value Optimization (NRVO)**
 - In the *initialization of an object*, when the source object is a nameless temporary and is of the same class type (ignoring cv-qualification) as the target object. When the nameless temporary is the operand of a return statement, this variant of copy elision is known as **Return Value Optimization (RVO)**
 - Return value optimization is mandatory and no longer considered as copy elision since **C++17**



Sorting Objects

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

Sorting Objects



Sorting Objects

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- To illustrate the effect by copy optimization, we consider a tiny sorting project
- We intend to sort objects of a data class `D` having resource of a class `R`
- We define the following to get started:
 - Resource class `R`
 - Data class `D`
 - A template function `swap`
 - A template function `sort` to bubble sort an array
 - The `main` function to initialize an array and sort it
- We are interested to see the trade-off of move and copy. So we build a statistics support in the code to count the number of constructions and destructions of the resource objects from class `R`
- Initial version works with only Copy operations
- We next add move operations in Data class `D` and move support in `swap` function
- We compare the statistics to show the huge benefit accrued with the move semantics



Sorting Objects: Copy Support

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

Sorting Objects: Copy Support



Resource Class, R

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- Let us consider a resource class **R**, and
- A data class **D** having resource **R**:

```
struct R { // Resource class - wrapper of int
    int i; // Wrapped resource
    R(int i) : i(i) { } // Parametric constructor
    R(const R& r) : i(r.i) { } // Copy constructor
    ~R() {} // Destructor
};

struct D { // Data class with resource
    R* r; // Resource to be dynamically constructed / destructed

    // ...
};
```



Data Class, D

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

```
struct D { // Data class with resource
    R* r; // Resource to be dynamically constru. / destru.
    D() : r(nullptr) { } // Default constructor - null resource
    D(int i) : r(new R(i)) { } // Parametric constructor - create resource
    D(const D& d) : r(new R(*(d.r))) { } // Copy constructor - copy resource
    D& operator=(const D& d) { // Copy assignment - copy resource
        if (this != &d) { // Self copy guard
            delete r; // Free resource
            r = new R(*(d.r)); // Copy resource
        }
        return *this;
    }
    ~D() { delete r; } // Destructor - free resource
    friend bool operator>(const D& c1, const D& c2) { // Compare D objects for sorting
        return c1.r->i > c2.r->i;
    }
    friend std::ostream& operator<<(std::ostream& os, const D& d) { // Stream D objects
        os << d.r->i << ' ';
        return os;
    }
};
```



sort Function

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- We store **N** number of **D** objs in an array
- We sort the array by Bubble Sort in ascending order

```
template<typename T>
void swap(T& a, T&b) { // Swap a and b using copy
    T t = a; // t copy-created from a: two a's
    a = b;    // a copy-assigned from b: two b's, one a destroyed
    b = t;    // b copy-assigned from t: two t's, one b destroyed
} // t destroyed

template<typename T>
void sort(T arr[], int n) { // Bubble Sort for easy analysis
    for (int i = 0; i < n - 1; ++i)
        for (int j = 0; j < n - i - 1; ++j)
            if (arr[j] > arr[j + 1]) { // Compare by D::operator>
                swap(arr[j], arr[j + 1]); // 3 constr.s and destr.s of R objs with copy
                                          // 0 constr. and destr. of R objs with move
            }
    }
```



main Function

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

```

int main() { // To populate and sort an array of D objs having R obj resources
    const int N = 10;           // Size of array and number of elements
    D arr[N];                   // Defa. initialization of array - use D::D() calls N times

    // Assignments of array elements with D objs having R obj resources
    // Fill with a strictly decreasing sequence for worst case of Bubble Sort
    for (int i = N - 1; i >= 0; --i)
        arr[i] = D(N - i); // Construct by D::D(int), assign by D::operator=(const D&)
                           // construct / destruct R objs
    for (int i = 0; i < N; ++i) // Print array before sorting. 10 9 8 7 6 5 4 3 2 1
        std::cout << arr[i]; std::cout << std::endl;
    sort(arr, N);              // Sort array in ascending order
    for (int i = 0; i < N; ++i) // Print array after sorting. 1 2 3 4 5 6 7 8 9 10
        std::cout << arr[i]; std::cout << std::endl;
}

```

- To get an estimate for the resource construct. and destruct., we build a worst-case for Bubble Sort, that is, populate `arr` in strictly descending order. Being sorting, this is dominated by `swap`
- Clearly in the worst case number of `swaps` = $\sum_{i=1}^{N-1} i = \frac{N*(N-1)}{2}$. Hence number of (unnecessary) resource constructions and destructions = $3 * \# \text{ of } \text{swaps} = \frac{3*N*(N-1)}{2}$



Sorting Objects: Statistics Support

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

Sorting Objects: Statistics Support



Resource Class R with Statistics

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- To count the exact number of constructions and destructions of **R** objects, we add three static counters in **R**
- We also add a static method **stat()** to print the statistics at anytime from anywhere

```
struct R { // Resource class
    int i; // Wrapped resource

    R(int i) : i(i) { ++nCtor; } // Parametric constructor
    R(const R& r) : i(r.i) { ++nC_Ctor; } // Copy constructor
    ~R() { ++nDtor; } // Destructor

    static unsigned int nCtor; // Count of direct construction of R objects
    static unsigned int nC_Ctor; // Count of copy construction of R objects
    static unsigned int nDtor; // Count of destruction of R objects

    static void stat(std::string s) { // Print R object statistics
        std::cout << s /* Banner message */ << "R obj Created = " << R::nCtor <<
            " R obj Copy Created = " << R::nC_Ctor << " R obj Destroyed = " << R::nDtor <<
            std::endl;
    }
};
```

Programming in Modern C++



Resource Class R with Statistics

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- Static counters of `R` are globally instantiated and initialized with 0's.
- We also add helper class `Stat` whose constructor and destructor calls `R::stat()`. Next we globally instantiate an object `extremeStat` of `Stat`
 - Being global static, `extremeStat` is constructed before `main()` is called and is destructed after `main()` returns
 - Hence the statistics are printed before calling `main()` and after returning from `main()`

```
// Instantiations of static R objects in global namespace
unsigned int R::nCtor = 0; // Count of direct construction of R objects
unsigned int R::nC_Ctor = 0; // Count of copy construction of R objects
unsigned int R::nDtor = 0; // Count of destruction of R objects

struct Stat { // Helper class to print R objects statistics
    Stat() { R::stat("Program Start: "); } // Construct before main(), initial stat
    ~Stat() { R::stat("Program End: "); } // Destruct after main(), final stat
} extremeStat;
```



main Function with Statistics

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

```
int main() { // To populate and sort an array of D objs having R obj resources
    const int N = 10;           // Size of array and number of elements
    D arr[N];                   // Defa. initialization of array - use D::D() calls N times
    R::stat("Array Defa: ");    // Statistics after Defa. initialization of array
    // Assignments of array elements with D objs having R obj resources
    // Fill with a strictly decreasing sequence for worst case of Bubble Sort
    for (int i = N - 1; i >= 0; --i)
        arr[i] = D(N - i); // Construct by D::D(int), assign by D::operator=(const D&)
                           // construct / destruct R objs
    R::stat("Array Init: ");    // Statistics after assignment of array
    for (int i = 0; i < N; ++i) // Print array before sorting
        std::cout << arr[i]; std::cout << std::endl;
    sort(arr, N);              // Sort array in ascending order
    R::stat("Array Sort: ");    // Statistics after sorting of array
    for (int i = 0; i < N; ++i) // Print array after sorting
        std::cout << arr[i]; std::cout << std::endl;
} // Statistics after destruction of array elements by s.~Stat()
```

Program Start: R obj Created = 0 R obj Copy Created = 0 R obj Destroyed = 0

Array Defa: R obj Created = 0 R obj Copy Created = 0 R obj Destroyed = 0

Array Init: R obj Created = 10 R obj Copy Created = 0 R obj Destroyed = 0

10 9 8 7 6 5 4 3 2 1

Array Sort: R obj Created = 10 R obj Copy Created = 135 R obj Destroyed = 135

1 2 3 4 5 6 7 8 9 10

Program End: R obj Created = 10 R obj Copy Created = 135 R obj Destroyed = 145



Analysis of Statistics

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- **Program Start:** R obj Created = 0 R obj Copy Created = 0 R obj Destroyed = 0
 - Static object `extremeStat` constructed by `Stat::Stat()` before `main()` is invoked, reports statistics
- **Array Defa:** R obj Created = 0 R obj Copy Created = 0 R obj Destroyed = 0
 - `D arr[N]: N = 10` D objects are constructed by `D::D()`. As `D::r` is set to `nullptr` in each, no R object is constructed
- **Array Init:** R obj Created = 10 R obj Copy Created = 10 R obj Destroyed = 10
 - `... = D(N - i): N = 10` D objects are constructed by `D::D(int)`. As `D::r` is set to `new R(i)` in each, `N = 10` R object is constructed
 - `arr[i] = ...: D(N - i)` is now *copy assigned* to `arr` elements by `D::D(const D& d)`. Hence, the resource R objects is destroyed (`delete r`) and constructed (`new R(*(d.r))`) for each
 - Note that `D(N - i)` is an *rvalue*, yet it is *copy assigned* as there is no *move assignment*
- **10 9 8 7 6 5 4 3 2 1**
 - `arr` before sorting. Filled with a strictly decreasing sequence
- **Array Sort:** R obj Created = 10 R obj Copy Created = 145 R obj Destroyed = 145
 - `sort(arr, N);` Being the worst case of bubble sort, $\frac{3*N*(N-1)}{2} = \frac{3*10*(10-1)}{2} = 135$ R objects are constructed by `R::R(const R&)` and destructed by `R::~~R()` for 45 swaps. Note that `t`, `a`, and `b` in `swap` are *lvalues*
- **1 2 3 4 5 6 7 8 9 10**
 - `arr` after sorting in increasing order
- **Program End:** R obj Created = 10 R obj Copy Created = 145 R obj Destroyed = 155
 - `int main() { ... }`: Remaining `N = 10` D objects destructed by `D:: D()` with `delete D::r`. Static object `extremeStat` destructed by `Stat::~~Stat()` after `main()` returns reports



Sorting Objects: Move Support

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

Sorting Objects: Move Support



Data Class D with Move Support

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- To minimize copies, we provide move operations in class **D** to be able to move **rvalues** whenever possible

```

struct D { // Data class with resource
    R* r; // Resource to be dynamically constru. / destru.
    D(); // Default constructor - null resource
    D(int i); // Parametric constructor - create resource
    D(const D& d); // Copy constructor - copy resource
    D& operator=(const D& d); // Copy assignment - copy resource
    ~D(); // Destructor - free resource

    D(D&& d) : r(d.r) { d.r = nullptr; } // Move constructor - move resource, ownership
    D& operator=(D&& d) { // Move assignment - move resource, ownership
        if (this != &d) { // Self move guard
            r = d.r; // Move resource
            d.r = nullptr; // Take ownership
        }
        return *this;
    }
};

```

- We again run the program and gather statistics



Analysis of Statistics: Move Support in Class D

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- Here is the statistics with move support. We note the changes:
- Program Start: R obj Created = 0 R obj Copy Created = 0 R obj Destroyed = 0
- Array Defa: R obj Created = 0 R obj Copy Created = 0 R obj Destroyed = 0
- Array Init: R obj Created = 10 R obj Copy Created = 0 R obj Destroyed = 0
 - `... = D(N - i):` $N = 10$ D objects are constructed by `D::D(int)`. As `D::r` is set to `new R(i)` in each, $N = 10$ R object is constructed
 - `arr[i] = ...: D(N - i)` is now *move assigned* to `arr` elements by `D::D(D&& d)` since `D(N - i)` is a *rvalue*
 - Hence, no resource R object is destructed or constructed - just ownership of resource is transferred
- 10 9 8 7 6 5 4 3 2 1
- Array Sort: R obj Created = 10 R obj Copy Created = 135 R obj Destroyed = 135
- 1 2 3 4 5 6 7 8 9 10
- Program End: R obj Created = 10 R obj Copy Created = 135 R obj Destroyed = 145



swap Function with Move Support

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- To minimize copies further, we provide move support in `swap()` function using `std::move`

```
template<typename T>
void swap(T& a, T&b) { // Swap a and b using move
    T t = std::move(a); // t move-created from a: a's ownership transferred to t
    a = std::move(b);   // a move-assigned from b: b's ownership transferred to a
    b = std::move(t);   // b move-assigned from t: t's ownership transferred to b
} // t destroyed, but no resource destruction as t had no ownership
```



Analysis of Statistics: Move Support in Class D and Function swap

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- Here is the statistics with move support. We note the changes:
- Program Start: R obj Created = 0 R obj Copy Created = 0 R obj Destroyed = 0
- Array Defa: R obj Created = 0 R obj Copy Created = 0 R obj Destroyed = 0
- Array Init: R obj Created = 10 R obj Copy Created = 0 R obj Destroyed = 0
- 10 9 8 7 6 5 4 3 2 1
- Array Sort: R obj Created = 10 R obj Copy Created = 0 R obj Destroyed = 0
 - `sort(arr, N);`: Being the worst case of bubble sort, $\frac{N*(N-1)}{2} = \frac{10*(10-1)}{2} = 45$ swaps are performed. But no swap copies any R object *only moves*
 - Hence no unnecessary construction and destruction of R objects
- 1 2 3 4 5 6 7 8 9 10
- Program End: R obj Created = 10 R obj Copy Created = 0 R obj Destroyed = 10



Sorting Objects: Summary

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

Sorting Objects: Summary



Analysis of Statistics: Summary

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

struct D		void swap(T&, T&)		R(int)	R(const R&)	~R()
Only Copy	Copy+ Move	Copy	Move			
Yes		Yes		N	$\frac{3N*(N-1)}{2} + N$	$\frac{3N*(N-1)}{2} + 2N = \frac{N*(3N+1)}{2}$
Yes			Yes	N	$\frac{3N*(N-1)}{2} + N$	$\frac{3N*(N-1)}{2} + 2N = \frac{N*(3N+1)}{2}$
	Yes	Yes		N	$\frac{3N*(N-1)}{2}$	$\frac{3N*(N-1)}{2} + N = \frac{N*(3N-1)}{2}$
	Yes		Yes	N	0	N

- With move support in the class and in swap function, we can elide $O(N^2)$ copies (and destructions)



Sorting Objects: Project Codes

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

Sorting Objects: Project Codes



Resource Class R

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

```

struct R { // Resource class
    int i; // Wrapped resource
    R(int i) : i(i) { ++nCtor; } // Parametric constructor
    R(const R& r) : i(r.i) { ++nC_Ctor; } // Copy constructor
    ~R() { ++nDtor; } // Destructor

    static unsigned int nCtor; // Count of direct construction of R objects
    static unsigned int nC_Ctor; // Count of copy construction of R objects
    static unsigned int nDtor; // Count of destruction of R objects

    static void stat(std::string s) { // Print R object statistics
        std::cout << s /* Banner message */ << "R obj Created = " << R::nCtor <<
            " R obj Copy Created = " << R::nC_Ctor << " R obj Destroyed = " << R::nDtor <<
            std::endl;
    }
};

// Instantiations of static R objects in global namespace
unsigned int R::nCtor = 0; // Count of direct construction of R objects
unsigned int R::nC_Ctor = 0; // Count of copy construction of R objects
unsigned int R::nDtor = 0; // Count of destruction of R objects

struct Stat { // Helper class to print R objects statistics
    Stat() { R::stat("Program Start: "); } // Construct before main(), initial stat
    ~Stat() { R::stat("Program End: "); } // Destruct after main(), final stat
} extremeStat;

```

Programming in Modern C++



Data Class D

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

```

struct D { // Data class with resource
    R* r; // Resource to be dynamically constru. / destru.
    D() : r(nullptr) { } // Default constructor - null resource
    D(int i) : r(new R(i)) { } // Parametric constructor - create resource
    D(const D& d) : r(new R(*(d.r))) { } // Copy constructor - copy resource
    D& operator=(const D& d) { // Copy assignment - copy resource
        if (this != &d) { // Self copy guard
            delete r; /* Free resource */ r = new R(*(d.r)); // Copy resource
        } return *this;
    }
    ~D() { delete r; } // Destructor - free resource
#ifdef _MOVE_ // If _MOVE_ is defined (set -D=_MOVE_ flag in GCC to define _MOVE_), use move operations
    D(D&& d) : r(d.r) { d.r = nullptr; } // Move constructor - move resource, ownership
    D& operator=(D&& d) { // Move assignment - move resource, ownership
        if (this != &d) { // Self move guard
            r = d.r; /* Move resource */ d.r = nullptr; // Take ownership
        } return *this;
    }
#endif // _MOVE_ // End of conditional compilation by _MOVE_
    friend bool operator>(const D& c1, const D& c2) { // Compare D objects for sorting
        return c1.r->i > c2.r->i;
    }
    friend std::ostream& operator<<(std::ostream& os, const D& d) { // Stream D objects
        os << d.r->i << ' '; return os;
    }
}

```



swap & sort Functions

Tutorial T10

Partha Pratim Das

Objective & Outline

Optimizing C++11 Programs

Copy Elision

Copy Initialization

Return Value Optimization (RVO)

Language Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

```

#ifndef _MOVE_ // If _MOVE_ is not defined, use copy version
template<typename T>
void swap(T& a, T&b) { // Swap a and b using copy
    T t = a; // t copy-created from a: two a's
    a = b;   // a copy-assigned from b: two b's, one a destroyed
    b = t;   // b copy-assigned from t: two t's, one b destroyed
} // t destroyed
#else // If _MOVE_ is defined (set -D=_MOVE_ flag in GCC to define _MOVE_), use move version
template<typename T>
void swap(T& a, T&b) { // Swap a and b using move
    T t = std::move(a); // t move-created from a: a's ownership transferred to t
    a = std::move(b);   // a move-assigned from b: b's ownership transferred to a
    b = std::move(t);   // b move-assigned from t: t's ownership transferred to b
} // t destroyed, but no resource destruction as t had no ownership
#endif // _MOVE_ // End of conditional compilation by _MOVE_

template<typename T>
void sort(T arr[], int n) { // Bubble Sort for easy analysis
    for (int i = 0; i < n - 1; ++i)
        for (int j = 0; j < n - i - 1; ++j)
            if (arr[j] > arr[j + 1]) { // Compare by D::operator>()
                swap(arr[j], arr[j + 1]); // 3 constr.s and destr.s of R objs with copy
                                         // 0 constr. and destr. of R objs with move
            }
    }
}

```

Programming in Modern C++



main Function

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

```
int main() { // To populate and sort an array of D objs having R obj resources
    const int N = 10;           // Size of array and number of elements
    D arr[N];                   // Defa. initialization of array - use D::D() calls N times
    R::stat("Array Defa: ");    // Statistics after Defa. initialization of array

    // Assignments of array elements with D objs having R obj resources
    // Fill with a strictly decreasing sequence for worst case of Bubble Sort
    for (int i = N - 1; i >= 0; --i)
        arr[i] = D(N - i); // Construct by D::D(int), assign by
                           // D::operator=(const D&) for copy, constr. / destru. R objs
                           // D::operator=(D&&) for move, no constr. / destru. R objs

    R::stat("Array Init: ");    // Statistics after assignment of array
    for (int i = 0; i < N; ++i) // Print array before sorting
        std::cout << arr[i];
    std::cout << std::endl;

    sort(arr, N);              // Sort array in ascending order

    R::stat("Array Sort: ");    // Statistics after sorting of array
    for (int i = 0; i < N; ++i) // Print array after sorting
        std::cout << arr[i];
    std::cout << std::endl;
} // Statistics after destruction of array elements by s.~Stat()
```



Problems

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- Provide construction / destruction counting and statistics generation support for class `D`
- Consider that the resource in `D` is held as a data member (`R r;`) and not as a pointer (`R *r;`). Provide appropriate support in classes `R` and `D` to avoid unnecessary copies during sorting
- Explore the move support in standard library containers, especially `vector` and `map`



Tutorial Summary

Tutorial T10

Partha Pratim
Das

Objective &
Outline

Optimizing
C++11 Programs

Copy Elision

Copy Initialization

Return Value
Optimization (RVO)

Language
Specification

Sorting Objects

Copy Support

Statistics Support

Move Support

Summary

Project Codes

Problems

Tutorial Summary

- Understood optimization by copy elision
- Understood copy / move optimization by Rvalues and Move Semantics
- Developed a complete sorting project with copy optimization by move